University of Hertfordshire

Department of Physics, Engineering and Computer Science

BSc Computer Science (Applied Data Science) Online

Data Science Project

Supervisor: Michael XXXXX

Final Project Report

by

Kamal Hassan

Student ID: XXXXXX

August 21, 2026

Investigating Key Factors Associated with Customer Subscription to a Term Deposit Using the Portuguese Bank Marketing Dataset

ACKNOWLEDGEMENTS

I would like to express my sincere appreciation to my project supervisor, Michael XXXXX, and module leader Mike XXXXX for the guidance, feedback and support provided throughout this project. The advice received during the planning, technical development and report-writing stages contributed significantly to the completion of the study.

I would also like to acknowledge the University of Hertfordshire for providing the academic resources and learning environment required to undertake this project. Finally, I am grateful to my family, friends and colleagues for their encouragement, patience and support throughout the research and development process.

ABSTRACT

This project explored the factors associated with customer subscription to a term deposit scheme and the potential of using machine learning for effective bank-marketing targeting. The Portuguese Bank Marketing dataset, containing 41.188 observations and 21 variables, was analysed using the Cross-Industry Standard Process for Data Mining methodology. Exploratory analysis revealed significant class imbalance, 12 duplicate observations and a number of categorical unknowns. Data preparation dropped duplicates, dropped the problematic pdays feature and generated a cleaned data set with 19 predictors and 41.176 observations.

The data were divided into stratified training, validation and test subsets using a 60/20/20 allocation. Algorithm specific preprocessing pipelines were created and SMOTE was limited to the appropriate training folds. Five-fold cross validation and GridSearchCV were used to train and optimise Logistic Regression, Random Forest and XGBoost. The accuracy, precision, recall, F1-score, and ROC-AUC were utilised to measure performance.

XGBoost achieved the highest reported test F1-score of 0.5091 and test accuracy of 0.8817. Random Forest had the best test ROC-AUC score of 0.7977, and logistic regression had the best test recall score of 0.6487. SHAP analysis revealed that the top 4 predictors are *"nr.employed", "contact", "emp.var.rate"* and *"poutcome".* The results show that campaign history and contact method and macro-economic conditions can help to provide more targeted customer targeting. However, external validation and live operational testing are required before practical deployment.

Keywords: bank marketing; term deposit; machine learning; customer targeting; XGBoost; SHAP; CRISP-DM

Table of Contents

ABSTRACT....3
LIST OF FIGURES....8
LIST OF TABLES....9
LIST OF ABBREVIATIONS....10
Chapter 1: Introduction....11
1.1 Background to the Study....11
1.2 Problem Statement....11
1.3 Project Aim....12
1.4 Project Objectives....12
1.5 Research Questions....13
1.6 Project Scope....13
1.7 Dissertation Structure....14
Chapter 2: Literature Review....15
2.1 Introduction....15
2.2 Bank Direct Marketing and Subscription Prediction....15
2.3 Machine Learning in Bank-Marketing Research....16
2.4 Classification Algorithms....17
2.4.1 Logistic Regression....17
2.4.2 Random Forest....17
2.4.3 XGBoost....17
2.5 Imbalanced Classification and Performance Evaluation....18
2.6 Explainable Artificial Intelligence....18
2.7 Research Gap....19
2.8 Chapter Summary....19
Chapter 3: Requirements....20
3.1 Introduction....20
3.2 Functional Requirements....20
3.3 Non-Functional Requirements....21
3.4 Project Constraints....21
3.5 Ethical and Legal Requirements....22
3.6 Chapter Summary....22
Chapter 4: Design....23
4.1 Introduction....23
4.2 Overall System Architecture....23
4.3 CRISP-DM Methodology....24
4.4 Experimental Design....25
4.5 Machine-Learning Pipeline Design....27
4.6 Software Design....27
4.7 Chapter Summary....28

Chapter 5: Technical Development....29
5.1 Introduction....29
5.2 Data Acquisition and Initial Inspection....29
5.3 Data Preprocessing....30
5.3.1 Cleaning and Feature Engineering....30
5.3.2 Dataset Partitioning....31
5.3.3 Algorithm-Specific Preprocessing....31
5.4 Machine-Learning Model Development....32
5.5 Implementation Verification....33
5.6 Chapter Summary....33
Chapter 6: Evaluation....34
6.1 Introduction....34
6.2 Evaluation Criteria....34
6.3 Cross-Validation and Hyperparameter Optimisation....35
6.4 Validation and Test Protocol....35
6.5 Model-Comparison Approach....36
6.6 Explainability Evaluation....37
6.7 Evaluation Limitations....37
6.8 Chapter Summary....37
Chapter 7: Results....38
7.1 Introduction....38
7.2 Exploratory Data Analysis Results....38
7.2.1 Dataset Structure....38
7.2.2 Target Distribution....39
7.2.4 Distributions and Relationships....40
7.3 Preprocessing Results....41
7.4 Model Results....41
7.4.1 Logistic Regression....41
7.4.2 Random Forest....41
7.4.3 XGBoost....42
7.4.4 Cross-Model Comparison....42
7.5 SHAP Results....42
7.6 Chapter Summary....43
Chapter 8: Discussion....44
8.1 Introduction....44
8.2 RQ1: Influential Customer, Financial and Campaign Factors....44
8.3 RQ2: Predicting Customer Subscription with Machine Learning....45
8.4 RQ3: Comparative Model Performance....46
8.5 RQ4: Support for More Effective Marketing Targeting....46

8.6 Ethical and Practical Implications....47
8.7 Limitations....48
8.8 Chapter Summary....48
Chapter 9: Conclusion....49
9.1 Project Summary....49
9.2 Principal Conclusions....49
9.3 Project Contribution and Achievement of Objectives....50
9.4 Recommendations....51
9.5 Future Work....51
9.6 Final Closing Statement....52
Reference List....53
Appendix A: Software Artefact Implemented....58
Appendix B: Complete EDA outputs....71
Appendix C: Complete preprocessing outputs....87
Appendix D: Machine-Learning Outputs....108
Appendix E: Testing and Verification Evidence....114
Appendix F: Risk Register....116
Appendix G: Project Milestone Table....117

LIST OF FIGURES

Figure 4.1: Overall Machine Learning Decision-Support System Architecture

Figure 4.2: CRISP-DM Phases Applied to This Project

Figure 4.3: Experimental Design Workflow

Figure 4.4: Machine Learning Pipeline Workflow

Figure 5.1: Preprocessing Workflow Implemented for the Portuguese Bank Marketing Dataset

Figure 5.2: Shared Model-Development and Evaluation Framework

Figure B.8: Numerical Feature Distributions, Group A

Figure B.8.1: Numerical Feature Distributions, Group B

Figure B.8.2: Boxplots for Numerical Feature Outlier Detection, Group A

Figure B.8.3: Boxplots for Numerical Features, Group B

Figure B.9: Correlation Heatmap for Numerical Features

Figure B.10: Numerical Feature Differences by Subscription Outcome

Figure D.1: SHAP Feature Importance for XGBoost

Figure D.2: SHAP Summary Plot

Figure D.3: SHAP Dependence Plot for nr.employed

Figure D.4: SHAP Waterfall Plot for an Individual Prediction

LIST OF TABLES

Table 7.1: Dataset Structure and Feature Types

Table 7.2: Target Variable Distribution

Table 7.3: Summary of Data Quality Findings

Appendix Tables B.0 to B.11: Complete Exploratory Data Analysis Outputs

Appendix Tables C.1 to C.13: Complete Preprocessing Outputs

Appendix Tables D.1 to D.14: Complete Machine-Learning and SHAP Outputs

LIST OF ABBREVIATIONS

AUC: Area Under the Curve

CRISP-DM: Cross-Industry Standard Process for Data Mining

EDA: Exploratory Data Analysis

IQR: Interquartile Range

ML: Machine Learning

ROC: Receiver Operating Characteristic

ROC-AUC: Receiver Operating Characteristic Area Under the Curve

SHAP: SHapley Additive exPlanations

SMOTE: Synthetic Minority Over-sampling Technique

UCI: University of California, Irvine

XAI: Explainable Artificial Intelligence

CHAPTER 1: INTRODUCTION

1.1 Background

Direct marketing enables banks to engage with customers and promote financial products without being totally branch-based. A typical product marketed by the telephone campaign is a term deposit, where a customer deposits money at a bank for a specific term, for an agreed rate of return. Although term deposits can be a source of stable funds for banks, the key to success in the campaign is to reach out to the customers who are likely to subscribe to the scheme.

These customers are not easily identifiable, with the decision to subscribe possibly being guided by a variety of factors relating to the demographic, previous campaign results, method of contacting them and economic conditions. Traditional campaigns that send out tons of emails to customers (who may not be the best target audience) can be time-consuming for staff and have minimal return. Data-driven techniques have been proven to adequately differentiate potential subscribers from those who are not likely to become subscribers using the Portuguese Bank Marketing dataset (Moro, Cortez and Rita 2014a; Apampa 2016). Machine learning can analyze the relationships between many different variables and help to make more targeted customer selections. This can enhance the efficiency of campaigns and support banks in better utilizing customer data (Ładyżyński, Żbikowski and Gawrysiak, 2019; Herausen et al., 2024).

1.2 Problem Statement

Indiscriminate telephone marketing can incur additional costs, cause inconvenience to customers and hurt their desire to receive future marketing. It can also cause employees to make numerous unsuccessful phone calls before receiving a positive response. This is the case in the Portuguese Bank Marketing dataset. Only 4640 out of 41.188 observations were customers that subscribed to a term deposit, resulting in a positive response rate of about 11.27% (Moro, Rita and Cortez, 2014b).

This imbalance results in a marketing challenge and an imbalanced binary-classification challenge. A model could be off target if it predicted that the majority of the customers were not subscribers, but that the smaller group of potential subscribers were subscribers. Therefore, the precision, recall, F1-score and ROC-AUC should be considered when assessing the model's performance (Fernández et al., 2018). It is also crucial that an effective solution is accompanied by the factors affecting predictions, which helps the results of the solution to be used for practical marketing decisions (Xie et al., 2023).

1.3 Project Aim

This project aims to investigate the factors associated with customer subscription to a term deposit and determine, if machine learning approach can support more effective bank-marketing targeting.

1.4 Project Objectives

The project objectives are to:

1. Conduct a comprehensive literature review to establish the academic context and identify existing approaches to bank marketing prediction.
2. Perform exploratory data analysis (EDA) to identify patterns and relationships within the dataset.
3. Preprocess the dataset through data cleaning, categorical encoding, feature scaling, and appropriate handling of class imbalance.
4. Develop and compare multiple machine learning classification models to predict customer subscription.
5. Evaluate model performance using Accuracy, Precision, Recall, F1-score, and ROC-AUC metrics.
6. Identify and rank the most influential factors affecting subscription decisions and translate findings into actionable marketing recommendations.

1.5 Research Questions

1. RQ1: Which customer, financial, and campaign-related factors have the greatest influence on term deposit subscription?
2. RQ2: Can machine learning classification models predict customer subscription behaviour?
3. RQ3: Which machine learning classification model demonstrates the strongest predictive performance for term deposit subscription?
4. RQ4: Can predictive insights derived from machine learning support more effective marketing targeting strategies?

1.6 Project Scope

The project uses the publicly available Portuguese Bank Marketing dataset from the UCI Machine Learning Repository (Moro, Rita and Cortez, 2014b). It handles the problem of subscription prediction as a binary-classification problem and contains Logistic Regression, Random Forest and XGBoost. The software artefact is a Python Jupyter Notebook containing the data analysis, preprocessing, modelling, evaluation and SHAP stages. The project does not run on real-life customer data and doesn't involve deployment in a bank. Decision Tree and Streamlit are not considered implemented and hence not counted as project outputs.

1.7 Dissertation Structure

Chapter 2 reviews relevant literature. Chapter 3 defines the project requirements, while Chapter 4 presents the system design. Chapter 5 explains the technical development. The evaluation procedure is described in Chapter 6, and the findings are reported in Chapter 7. Chapter 8 discusses the findings in relation to existing research and their marketing implications. Chapter 9 presents the conclusions, recommendations, limitations and opportunities for future development.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

This chapter discusses the studies conducted on bank direct marketing, prediction of customer response and machine-learning classification. It takes into account the importance of predictive targeting, use of machine learning with bank-marketing data, advantages and disadvantages of the chosen algorithms, class imbalance, model explanation and model performance evaluation (Herhausen et al., 2024). The review ends with the research gap being filled in by this project.

2.2 Bank Direct Marketing and Subscription Prediction

Direct marketing allows banks to market products to their customers via any direct marketing medium including telephone, email and electronic communications. Direct marketing can be used by an organisation to target an individual and measure the response. However, its effectiveness is dependent upon the selection of those customers who are likely to buy the product being promoted. Without accurate targeting, contacting customers can be costly, time-consuming for employees and lead to a bad customer experience.

Moro, Laureano and Cortez (2011) demonstrated how the CRISP-DM methodology could be applied to bank direct marketing by connecting business objectives, customer data and predictive modelling. Subsequent studies with the Portuguese Bank Marketing (PBM) dataset have indicated that customer, campaign and economic information could be useful to predict the term-deposit subscription (Moro, Cortez and Rita, 2014a). Apampa (2016) further concluded that classification and ensemble techniques could be used in predicting bank-marketing response, but noted the performances of the models depended on the classification algorithm and evaluation metric used.

Predictive targeting can thus enable banks to focus on those customers who are more likely to respond. It can also help to make more personalised interactions if predictions are combined with responsible marketing judgement. This aligns with the broader trend towards data-driven marketing, where analytical systems help in customer selection and campaign planning (Herhausen et al., 2024).

2.3 Machine Learning in Bank-Marketing Research

Neural networks have been used in banking applications like customer-response prediction, credit assessment, customer retention, product recommendation etc. Supervised-learning algorithms are used to learn the relationship between predictor variables and a known response, in bank marketing. From this model, it can be predicted the likelihood of a new customer to subscribe.

Ładyżyński, Żbikowski and Gawrysiak (2019) demonstrated that machine learning could be used for targeting banking products, detecting patterns in information that is interesting for the targeted marketing. Several studies using bank-marketing datasets have investigated the comparison of individual classifiers and ensemble models, and it has been found that advanced classifiers have better predictive performance than simple baseline classifiers (Ganar and Hosein, 2022; Keshava et al., 2019). In order to better understand the contributors to telemarketing success, Xie et al. (2023) extended this idea and incorporated an interpretability analysis with prediction.

However, reported results are not directly comparable in every case. Studies may use different dataset versions, preprocessing decisions, data splits, balancing methods and performance measures. A model that achieves high accuracy in one experiment may not identify subscribers effectively if the positive class is underrepresented. Consequently, algorithm selection must be considered together with the experimental design and evaluation method.

2.4 Classification Algorithms

2.4.1 Logistic Regression

Logistic Regression is used to estimate the probability of a binary outcome, and is very common with binary classes for the target. The coefficients of this model offer a fairly straightforward understanding of the relationship between the predictor variables and the outcome, which makes this model useful for interpretation purposes (Harris, 2021). It will also serve as a baseline model to compare with more complex models. But its standard form assumes that there is primarily a linear association between the predictors and the log odds of the outcome. This can be restrictive in addressing customer behaviour with complex interactions or nonlinear patterns.

2.4.2 Random Forest

Random Forest is an ensemble technique which is based on many DTs trained with different samples and feature subsets. They are combined together so as to dampen the fluctuations that can be found with a single tree (Breiman, 2001). It is possible to encode nonlinear relationships and interactions, without having to be formulated beforehand. It is also fairly tolerant of noise and mixed patterns of features. These attributes make Random Forest suitable for the data from bank-marketing, however, unlike Logistic Regression, the combined structure is less directly interpretable.

2.4.3 XGBoost

The XGBoost is a gradient boosting algorithm that builds up the trees sequentially, where each new tree tries to fix the errors of the previous one. It features regularisation and computational enhancements which make it efficient to learn from structured data sets (Chen and Guestrin, 2016). Since XGBoost can capture complex relationships and interactions, it has been successfully used in the prediction of customer responses. Combining such predictive approaches with interpretation techniques can be beneficial in offering not only performance information but also beneficial marketing insight, Xie et al. (2023) concluded. However, to prevent overfitting, there is a need for careful hyperparameter optimization and validation of XGBoost.

2.5 Imbalanced Classification and Performance Evaluation

The Portuguese Bank Marketing dataset is imbalanced since the number of subscribers is significantly less than the number of non-subscribers. Imbalanced data is challenging to learn due to the possibility that a classifier might be biased towards the majority class while still getting apparently good accuracy (He and Garcia, 2009). Although techniques like SMOTE may help to better represent the minority class while training the model, care must be taken to not contaminate validation or test data (Fernández et al., 2018).

A number of measures are thus needed. Precision is the proportion of subscribers predicted to be subscribers and the proportion of recall is the proportion of those who subscribed predicted to be subscribers (Sokolova and Lapalme, 2009). The F1 score is a combination of precision and recall in

harmonic mean. ROC-AUC is a measure of the performance of a model in separating the two classes at various thresholds (Sokolova and Lapalme, 2009). From a practical point of view, imbalance strategies should be evaluated, as their effectiveness is related to the data, algorithm and desired outcome (Wainer, 2024). For this project, F1-score and ROC-AUC are given more importance and accuracy, precision and recall are considered as supporting evidence (Moro, Cortez and Rita, 2014a; Mishra et al., 2026).

2.6 Explainable Artificial Intelligence

Predictive performance isn't the only reason for assigning a customer to a class. In the banking sector, it is crucial because marketing suggestions ought to be clear and subject to review. SHAP relies on Shapley values to determine the contribution of each feature to a prediction, compared to the model's baseline prediction (Lundberg and Lee, 2017). It can offer feature rankings and explanations of individual predictions across the globe. However, Xie et al., (2023) showed its usefulness in the combination of prediction and interpretation in the analysis of bank telemarketing. SHAP is used for the task of identifying influential factors upon the evaluation of the model, which in this project are the customer, campaign and macroeconomic factors.

2.7 Research Gap

Existing literature on bank-marketing shows that machine-learning based prediction of customer responses is possible, but much of it focuses on predictive performance rather than the entire chain from data preparation to fair evaluation to explanation and business use. Some of the reported comparisons are also not easily interpretable because of the differences in preprocessing and evaluation.

This project aims to fill this gap by incorporating leakage-controlled preprocessing, multiple evaluation measures, comparing Logistic Regression, Random Forest and XGBoost and SHAP interpretation. It also puts the analytical results into a marketing context rather than considering model performance as the sole project result.

2.8 Chapter Summary

Though literature justifies the idea of using machine learning for strategic bank marketing, there is also a discussion of problems related to class imbalance, evaluation and interpretability. The results of this research guide the functional, technical, ethical and performance specifications given in Chapter 3.

CHAPTER 3: REQUIREMENTS

3.1 Introduction

The aim of the project and the research question are translated into functional and non-functional, practical and ethical/legal requirements in this chapter. These requirements specify what the software artefact is supposed to do and the standards that it should meet. They also make sure that the implementation that will be completed is in line with the data that is available, the scope of the project and the purpose of the marketing.

3.2 Functional Requirements

System should load the Portuguese Bank Marketing dataset, and examine the dimensions, variables, data types and target distribution. It should be able to perform exploratory data analysis to find distributions, duplicate records, unusual categories, class imbalance and the relationships between predictors and subscription outcomes. The completed exploratory analysis can be found in Appendix B.

The data set needs to be cleaned and data transformed prior to modelling. Some necessary operations were removing the duplicates, excluding the duration variable to avoid target leakage, engineering the previously contacted feature and encoding the binary target. The data should then be split up into stratified training, validation and test subsets of 60/20/20. All algorithm-appropriate pre-processing of numerical and categorical variables must be performed on training data, and applied to the validation and test data sets. Preprocessing evidence is fully presented in Appendix C.

The implementation should include: Logistic Regression, Random Forest and XGBoost classifiers. For each model 5-fold cross validation and hyperparameter optimisation to be done using GridSearchCV. The accuracy, precision, recall, F1-score and ROC-AUC should be used to assess the performance. Then SHAP needs to be applied to the chosen model to find influential features. The outputs from machine learning is presented in Appendix D and the Python code to implement is provided in Appendix A. The final artefact should be a Jupyter Notebook that can be reproduced.

3.3 Non-Functional Requirements

Reproducibility of notebooks must be ensured by fixed random states, consistent data partition and ordered code. When using pipelines in Scikit-learn, it is important to have processing steps that are reliable and appropriate for modelling (Pedregosa et al., 2011). To prevent data integrity loss, only appropriate transformations should be applied to training data and the data sets should be verified before data modelling. This validation is crucial as the machine-learning systems' reliability can be compromised if data issues are not identified (Polyzotis et al., 2019).

The code should also be understandable, well-structured and broken into logical sections. The time of execution should be within an acceptable range in terms of the computing environment. Results need to be interpretable by a non-technical reader and SHAP results need to facilitate model explainability (Lundberg and Lee, 2017). Labelled sections of the notebook, clearly presented outputs and documented analytical steps are evidence of usability.

3.4 Project Constraints

This project will use a public source historical data set, and there is no access to current bank customers or any other organisational data. Very imbalanced target variable and limited computing resources (hyperparameter searching). The artefact is not tested under production of banking conditions. Thus, the results will be influenced by the length of time, the customers and the Portuguese marketing context of the data collected.

3.5 Ethical and Legal Requirements

The data does not need any recruitment of participants or any direct customer contact (Moro, Rita and Cortez, 2014b), so the data is anonymised secondary data. However, demographic characteristics need to be treated fairly and should not be used to justify discriminatory targeting. Predictions should not automatically bar the customer from decision making, but should assist human decision making.

The dataset needs to be properly cited and applied in line with the conditions of the repository. Whilst no identifiable personal data is being processed, the principles found in the Data

Protection Act 2018 should be considered in relation to the project; being fair, transparent, purpose limited and data minimised.

Jupyter Notebook built-in AI assistance was used to proofread developed Python code for the technical software artefact, including identifying syntax errors, debugging and formatting issues. The code implemented was reviewed prior to implementation, and all responsibility for the final artefacts remain with the author. The use of AI was conducted in accordance with University of Hertfordshire policies on the appropriate use of AI in academic work.

3.6 Chapter Summary

The requirements establish a reproducible, explainable and ethically responsible machine-learning artefact. Chapter 4 extrapolates them into the technical and experimental design of the project.

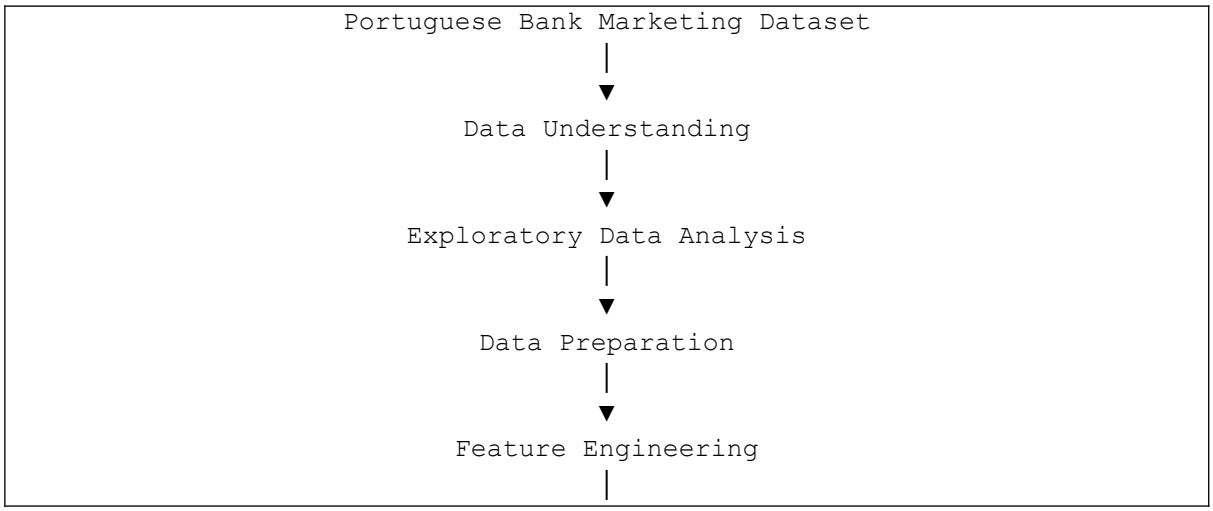
CHAPTER 4: DESIGN

4.1 Introduction

This chapter introduces the technical and experimental design of the machine-learning artefact, which is derived from the requirements defined in chapter 3. The design is based on the Cross-Industry Standard Process for Data Mining (CRISPDM), thus offering a structured way from understanding the business to interpreting the model and deploying it academically (Chapman et al., 2000). It includes the system architecture, methodological phases, experimental controls, machine-learning pipeline and software structure.

4.2 Overall System Architecture

The system was designed to be a sequential decision-support transforming raw campaign data into evidence for marketing targeting. The process starts by using the Portuguese Bank Marketing dataset from UCI Machine Learning Repository (Moro, Rita and Cortez, 2014b) as demonstrated in Figure 4.1. After understanding the data and conducting initial exploratory analysis, cleaning and algorithm specific pre-processing were performed (Chapman et al., 2000; Kaufman et al., 2012). Then Logistic Regression, Random Forest (RF) and XGBoost were trained and evaluated. SHAP interpretation of the selected model and interpretation of results in marketing recommendations. Such architecture will allow each analytical step to be traced back to the original data and project requirements.



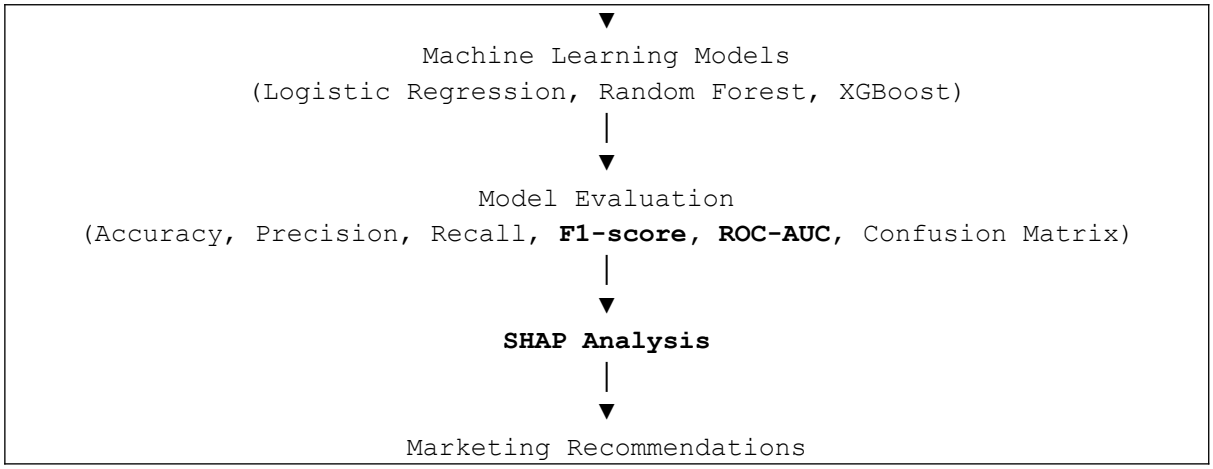


Figure 4.1: Overall Machine Learning Decision-Support System Architecture

4.3 CRISP-DM Methodology

CRISP-DM was chosen because it is related to a specific business issue, and it provides guidance for an iterative data-mining process (Moro, Laureano and Cortez, 2011; Chapman et al., 2000). Figure 4.2 shows how the six CRISP-DM phases were applied to this project.

Business understanding provides a description of the problem of inefficient bank-marketing targeting, the research question, and the practical contribution. Data understanding encompasses an exploration of the structure, feature types, distribution and quality of the target data set. Data preparation includes duplicate removal, leakage prevention, feature engineering, encoding and scaling, outlier treatment and class balancing techniques (Chu et al., 2016; Kaufman et al., 2012; Lemaître, Nogueira and Aridas, 2017).

Optimization and modeling of Logistic Regression, Random Forest and XGBoost is done in the modelling phase. Comparisons of the predictive performance of these models and an evaluation of generalisation to held-out data are performed (Varma and Simon, 2006; Bischl et al., 2023). The last phase is for academic deployment and not for operational deployment. These include the presentation of the reproducible notebook, model results, SHAP explanations and marketing recommendations. Such an application of CRISP-DM is similar to previous work on using it in the analysis of bank direct-marketing (Moro, Laureano and Cortez, 2011).

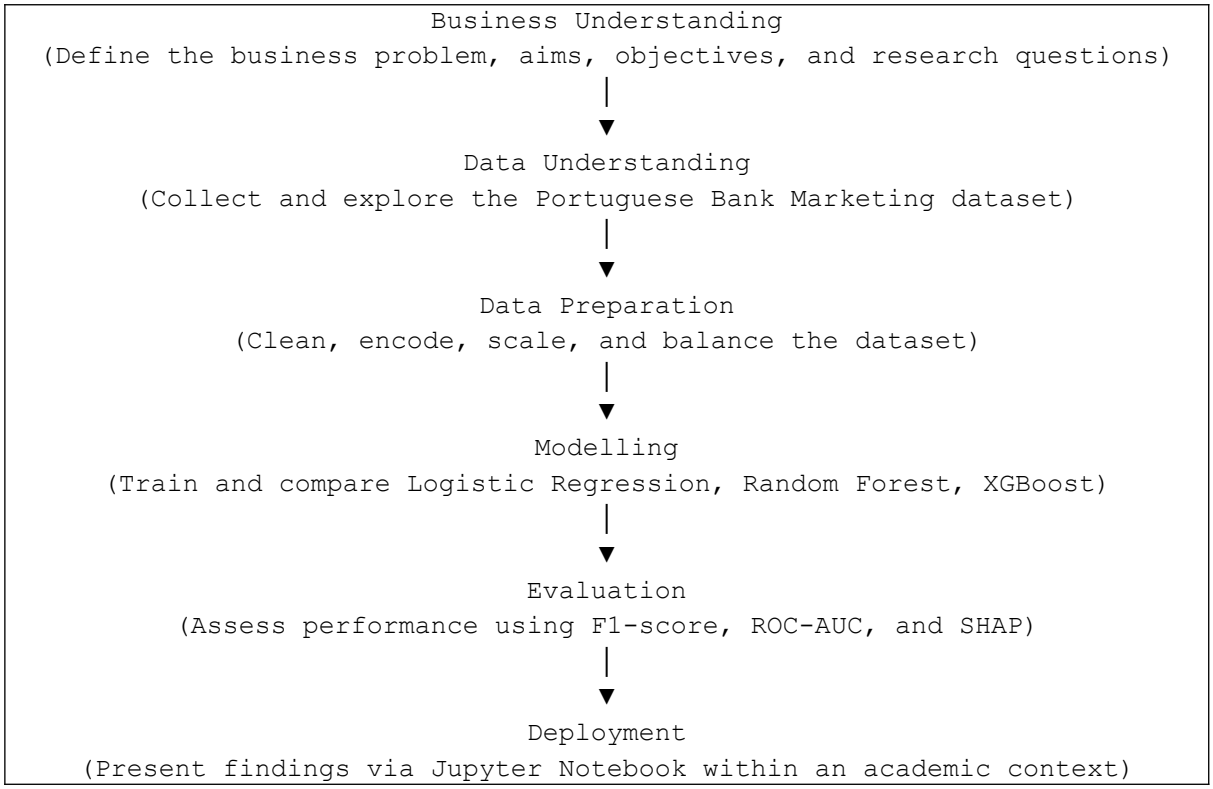


Figure 4.2: CRISP-DM Phases Applied to This Project

4.4 Experimental Design

The prediction task was structured as a binary classification problem with the target signifying client's decision to subscribe to a term deposit or not. The data was first cleaned and then feature engineered before being split into a 60/20/20 training, validation and test sets respectively, with stratified sampling. Minority subscription did not change between the three subsets due to stratification.

To avoid the risk of information from the validation and test data influencing the development of the model, the preprocessing parameters were learned from the training data. This control helps to minimize leakage and better reflect performance on unseen observations (Kaufman et al., 2012). SMOTE was only applied to the respective training folds. Hyperparameter optimisation was performed using five-fold cross validation and GridSearchCV and model comparison was done using the validation set. Held out test set gave evidence of final performance. Confidence in the performance is lowered due to the separation of the optimisation from the final test (Varma and

Simon, 2006; Bischl et al., 2023). Split and preprocessing outputs are shown in detail in Appendix Tables C.6 through C.13.

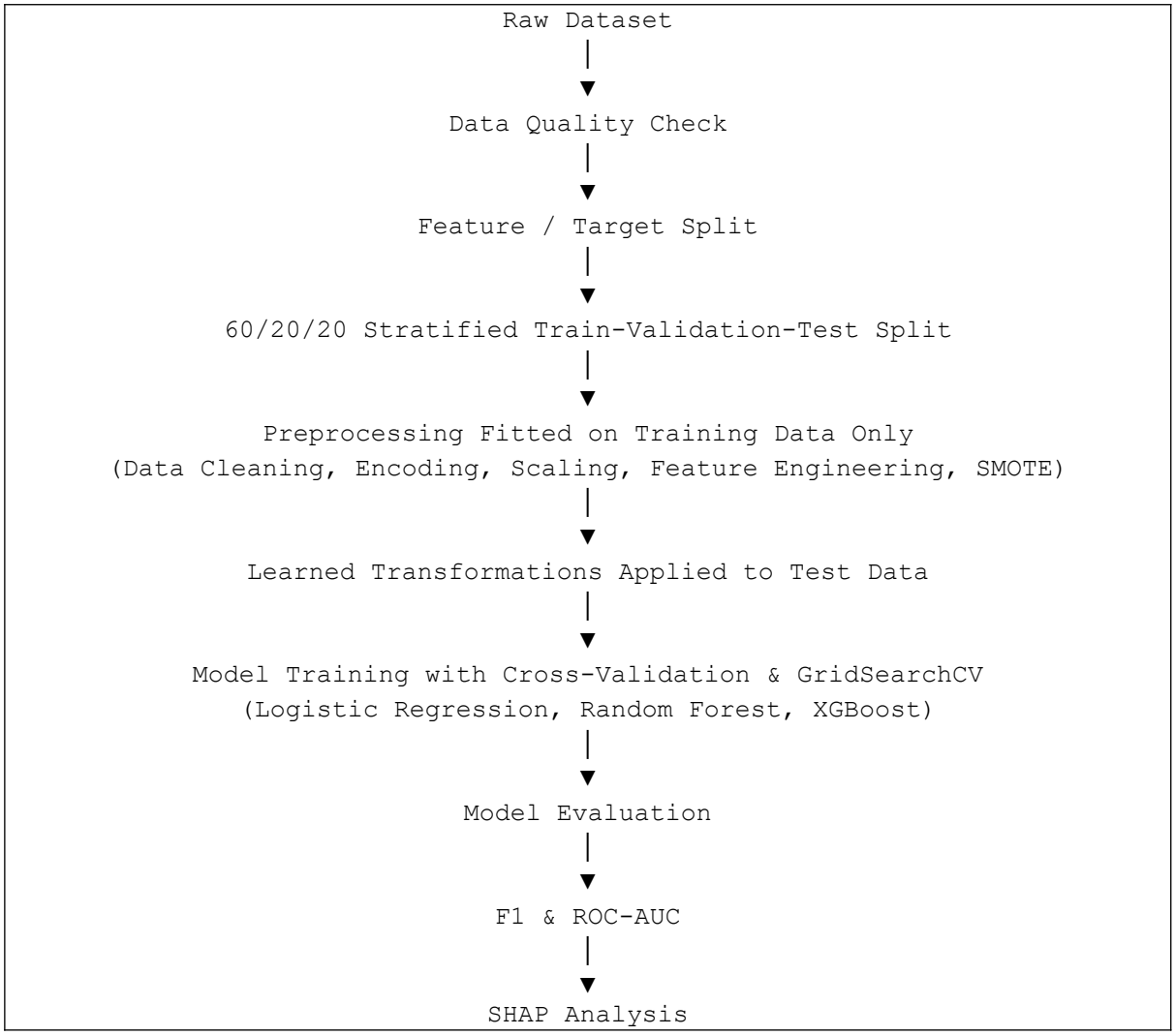


Figure 4.3: Experimental Design Workflow

4.5 Machine-Learning Pipeline Design

Figure 4.4 shows the sequence implemented for the library import and loading of the data, exploratory analysis, data cleaning, partitioning and preprocessing (Chapman et al., 2000). The pre-processed data then go through the model-development, optimisation and evaluation stages. The end stages involve SHAP and also marketing recommendations. A common sequence makes it easier to compare and allows for individual preprocessing for different algorithms. The implementation is given in Appendix A.

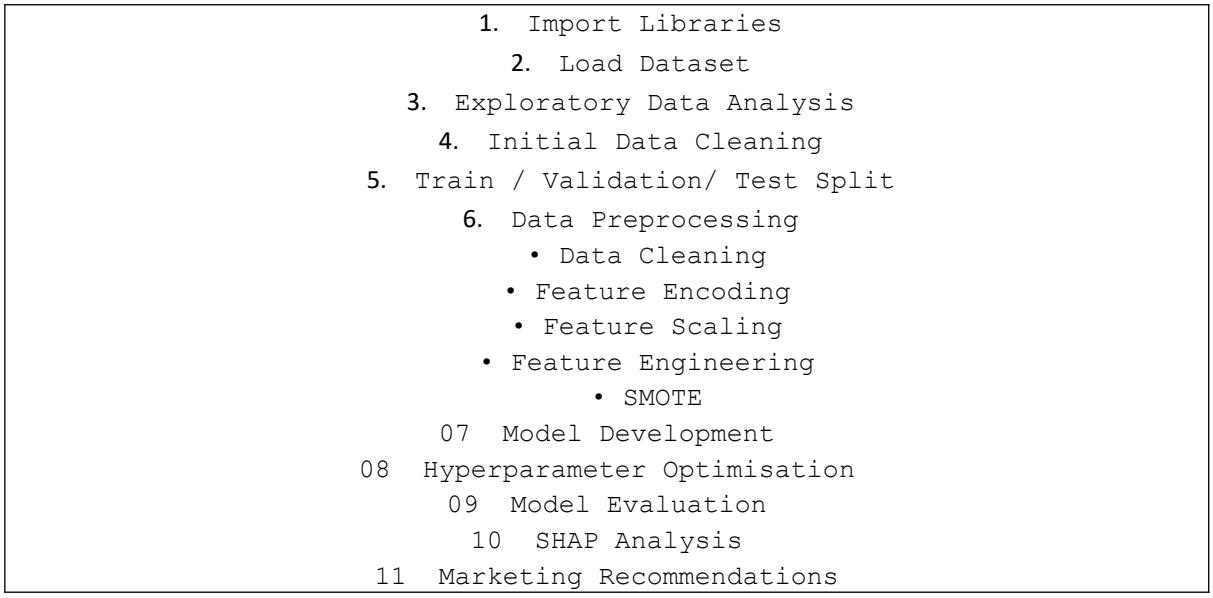


Figure 4.4: Machine Learning Pipeline Workflow

4.6 Software Design

The artefact has been realized as a modular python jupyter notebook. Data management, as well as preprocessing, pipelines, and cross-validation and model evaluation, are supported with Pandas and NumPy, respectively, while Scikit-learn is used for the preprocessing and pipelines functions (McKinney, 2010; Harris et al., 2020; Pedregosa et al., 2011). The gradient-boosting classifier (GBM) is implemented in XGBoost (Chen and Guestrin, 2016) and SHAP is used for supporting the model interpretation (Lundberg and Lee, 2017). Exploratory visualisations were created using Matplotlib and Seaborn (Hunter, 2007; Waskom, 2021). The notebook design makes it easy to implement each step, review and reproduce it in a logical sequence.

4.7 Chapter Summary

The design relates the needs to an architecture based on CRISP-DM, a controlled experiment and a reproducible software architecture. Chapter 5 explains how the design was implemented.

CHAPTER 5: TECHNICAL DEVELOPMENT

5.1 Introduction

In this chapter, the design from Chapter 4 will be implemented using Python in a Jupyter Notebook environment. The development steps included data acquisition, inspection, cleaning, feature engineering, dataset partition, algorithm specific pre-processing, model training and verification of model implementation (Chapman et al., 2000; Kaufman et al., 2012). The entire software artefact is included in Appendix A, and the analytical outputs are presented in Appendix B, Appendix C and Appendix D.

5.2 Data Acquisition and Initial Inspection

The notebook was loaded with the Portuguese Bank Marketing dataset, which is a structured dataset of 41.188 observations and 21 variables (Moro, Rita and Cortez, 2014b). An initial data quality inspection was carried out to check its dimensions, column names, data types and target categories. There were 10 numerical and 11 categorical variables in the dataset, one of which was the target variable y. The inspection also noted that the response was in the form of yes and no categories. Descriptive statistics, categorical frequencies, duplicate records, unknown categories, class distribution and numerical distributions and correlations were explored (Agarwal, Prabha and Yadav, 2025). There was a significant imbalance between classes, with just 4.640 observations being subscribers (before cleaning). It also found 12 duplicate records, and a few categorical unknown values, but there were no standard missing values. The implementation code is included in Appendix Listings A1.1 through A2.7, and the full set of data structure is provided in Appendix Table B.1. Figure 5.1 summarises the preprocessing workflow implemented before model development.

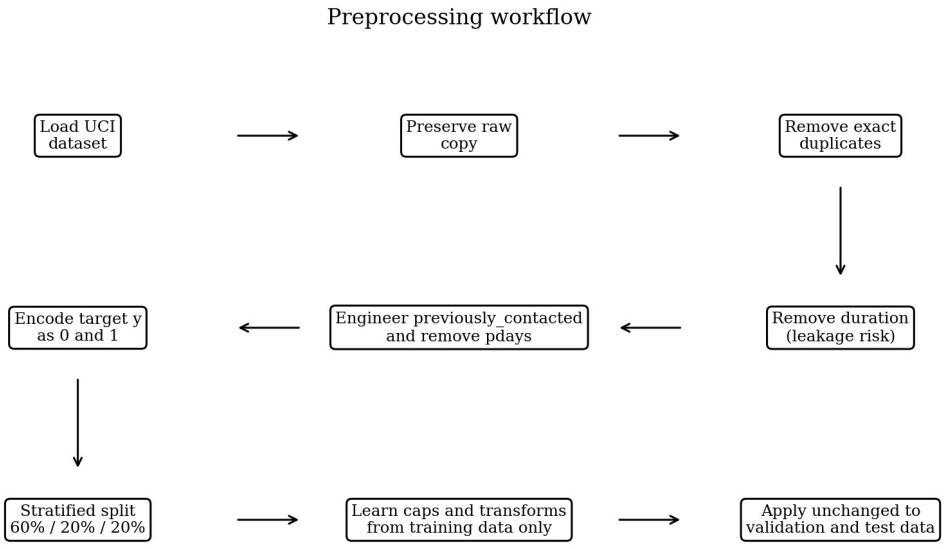


Figure 5.1: Preprocessing Workflow Implemented for the Portuguese Bank Marketing Dataset

5.3 Data Preprocessing

5.3.1 Cleaning and Feature Engineering

The first cleaning operation resulted in the removal of the 12 duplicate observations, resulting in a number of 41.176 records. The repeated observations were avoided to prevent unwanted effects on model training. Structured cleaning is crucial as unresolved data-quality issues may negatively impact the quality of subsequent analyses (Chu et al., 2016).

Afterwards, the *"duration"* variable was deleted. Only known after the contact has been made, this feature stores the length of the telephone call. Including it would introduce target leakage because call duration is unavailable when customers are selected for contact (Kaufman et al., 2012). The variable was therefore removed, consistent with the dataset documentation (Moro, Rita and Cortez, 2014b).

A customer with pdays of 999 meant that the customer wasn't reached previously. This information was captured by creating a binary variable previously_contacted, and then the original pdays variable was dropped (Moro, Rita and Cortez, 2014b). The target was set to 0 when it was not subscribed or 1 when it was subscribed. This resulted in 19 predictors and one binary target. The

evidence of cleaning and feature-engineering is provided in Appendix Tables C.1–C.5 and Appendix Listings A3.1–A3.6.

5.3.2 Dataset Partitioning

The cleaned data were split into training, validation and test sets of 60%, 20% and 20% respectively. The training set contained 24.705 observations, the validation set had 8.235 observations and the test set consisting of 8.236 observations. Approximately 11.27% of the positive-class was maintained in each subset after stratification. Further checks were made to ensure that was no overlap between the three subsets and that all records were covered (Kaufman et al., 2012; Varma and Simon, 2006). The split implementation and validation outputs can be found in Appendix Listings A3.7 and A3.8 and Appendix Tables C.6 and C.7.

5.3.3 Algorithm-Specific Preprocessing

There were 9 numerical and 10 categorical predictors. Age and campaign was capped using interquartile-range boundaries learned from the training data as outlier capping. Capping extreme observations instead of deletion without discarding any customer records, which is beneficial when extreme observations might be valid observations (Boukerche, Zheng and Alfandi, 2020).

Three branches for separate preprocessing were established due to the different requirements of the three algorithms. In Logistic Regression, categorical variables were one hot encoded and numerical variables were scaled resulting in 52 model inputs. Categorical variables were hot-encoded for Random Forest and XGBoost while numerical variables were kept in the same measurement units as needed with the corresponding cleaning and capping (Pedregosa et al., 2011; Chen and Guestrin, 2016). The ordinal approach created a consistent numerical representation, but was susceptible to the imposition of an artificial ordering on nominal categories.

All transformation rules were picked up from training data and were then applied to validation and test observations (Kaufman et al., 2012). The synthetic minority observations were not included in the validation and test folds to ensure that they were restricted to the folds in which they were created. SMOTE is capable of enhancing the minority class representation in the training process and can be used for training as learning from imbalanced data (Chawla et al., 2002; Lemaître,

Nogueira and Aridas, 2017). The full output of the preprocessing is available in Appendix Tables C.8 – C.13 and Appendix Listings A3.9 – A3.14.

5.4 Machine-Learning Model Development

A common experimental setup was used to develop Logistic Regression, Random Forest and XGBoost as illustrated in Figure 5.2. An interpretable base-learner was provided using Logistic Regression (Harris, 2021). Random Forest was an ensemble of multiple decision trees, which were used to capture nonlinear patterns, and to reduce the instability of an individual tree (Breiman, 2001). Sequential gradient-boosted trees with regularisation (XGBoost) were implemented to learn more complex relationships (Chen and Guestrin, 2016).

Both algorithms were cross-validated and hyper parameters were optimized with 5-fold cross validation and GridSearchCV. The strategy enabled to systematically compare candidate configurations, and decreased the reliance on a single training split, as noted by Bischl et al., (2023). The Logistic Regression code appears in Appendix Listings A4.1 - A4.4, Random Forest code provided in Appendix Listings A4.2.1 - A4.2.4 and XGBoost code can be found in Appendix Listings A4.3.1 - A4.3.4.

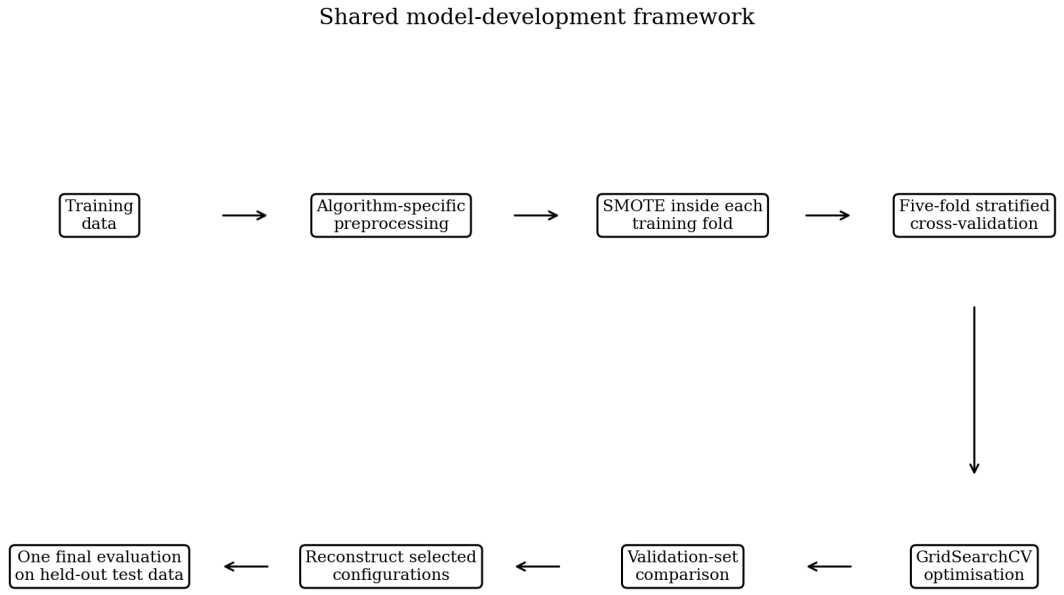


Figure 5.2: Shared Model-Development and Evaluation Framework

5.5 Implementation Verification

Checks for preprocessing branches to verify no values were missing, the correct feature dimensions were created and the three sets of data isolated separately (Polyzotis et al., 2019). Random states were fixed, allowing reproducibility. The outputs of each classifier were examined for training, validation, cross validation and test sets and comparative scoring and generalisation gaps were generated (Varma and Simon, 2006; Bischl et al., 2023) using Appendix Listing A5.1. The SHAP implementation used to interpret the selected XGBoost model, is listed in Appendix Listing A6.1. The associated implementation checks are documented in Appendix Listings A5.1 and A6.1.

5.6 Chapter Summary

The implementation turned the project design into a reproducible machine learning workflow that consisted of leakage control. It processed the dataset and created three classifiers and the evidence for evaluation and results chapters (see Appendix Listings A5.1 and A6.1).

CHAPTER 6: EVALUATION

6.1 Introduction

This chapter discusses how the completed machine-learning models were assessed. It concentrates on the criteria; the validation processes and the comparison rules utilised to evaluate the performance (Sokolova and Lapalme, 2009; He and Garcia, 2009). The numerical results are provided in Chapter 7. This separation prevents the evaluation methodology is confused with the interpretation of individual model results. This distinction follows the metric-based evaluation framework of Sokolova and Lapalme (2009).

6.2 Evaluation Criteria

There are a lot more non-subscribers than subscribers in the data set. For this reason, no single measure was considered sufficient. The accuracy is the ratio of the number of all observations correctly classified to the total number of observations, but it can be misleading if one class is much smaller than the other. With a model that has high accuracy, most customers are predicted as non-subscribers and few subscribers are predicted, but in fact, these few customers are subscribers (He and Garcia, 2009; Fernández et al., 2018).

Precision is the percentage of customers that actually subscribed to the ones that were predicted as subscribers. Precision can lower marketing resources spent on those customers who are not likely to respond. Recall: The percentage of actual subscribers that the model identified. The more people who can recall, the less amount of potential customers lost due to the campaign. However, higher recall, may lead to less precision due to more false positive predictions (Sokolova and Lapalme, 2009).

The F1-score takes into account precision and recall, by computing their harmonic mean. It was taken as a main measure since it represents a balance between correct detection of the subscribers and reduction of false positive predictions (Sokolova and Lapalme, 2009). ROC-AUC assesses the performance of a model to distinguish between subscribers and non-subscribers at various

classification levels. It thus offers an evaluation of discrimination that is independent of thresholds (Sokolova and Lapalme, 2009).

True positives, true negatives, false positives and false negatives were documented using confusion matrices. These measures in combination provided a complete assessment, other than accuracy. The practical significance of their use depends on the use of the model and the relative consequences of the misses and the unnecessary contacts (Wainer, 2024).

6.3 Cross-Validation and Hyperparameter Optimisation

Model building was done using five-fold cross validation. The training data has been split into five folds, each time using four folds for fitting the model and the fifth fold for evaluating the model. This was repeated until all of the folds were used as the evaluation fold. The mean performance and variation among the folds were then explored to determine stability (Varma and Simon, 2006; Bischl et al., 2023).

GridSearchCV tried out various combinations of hyperparameters and chose the best hyperparameter combination based on the highest cross validation F1 score. Though systematic optimisation can help minimize the need for arbitrary settings, an extensive search can overfit the development data (Bischl et al., 2023; Varma and Simon, 2006). Table D.2, D.3, D.6, D.7, D.10 and D.11 in Appendix provide the results of cross validation and hyperparameter tuning.

6.4 Validation and Test Protocol

Training subset was utilized to learn preprocessing parameters, to create SMOTE observations within training folds, and for model parameters. The validation subset was meant to provide a means for comparison after tuning, but not to be shown the test data. A test subset was set aside to offer a final measure of performance on observations never seen before.

Generalisation gaps between training, validation and test metrics were assessed (Varma and Simon, 2006). If the increase in performance is great, compared to the validation or test set, it could indicate overfitting. The same values for all three subsets would be proof that a model has learnt patterns which generalised beyond the observations it has seen (Varma and Simon, 2006). There were

potential sources of bias that were not eliminated by the use of separate datasets, but the use of separate datasets resulted in a stronger evidence assessment than the use of training data alone (Kaufman et al., 2012; Varma and Simon, 2006).

6.5 Model-Comparison Approach

F1-score and ROC-AUC were prioritised because they provide complementary evidence for imbalanced classification. F1-score balances precision and recall, whereas ROC-AUC measures the model's ability to distinguish subscribers from non-subscribers across classification thresholds (He and Garcia, 2009; Sokolova and Lapalme, 2009). Accuracy, precision and recall were retained as supporting measures.

Validation results were used to examine performance during development, while the final test results and the combined F1-score and ROC-AUC ranking are reported in Appendix Table D.13. XGBoost was identified as the reported final model because it achieved the highest test F1-score and combined ranking. However, the final choice was influenced by test-set comparisons, so the test set did not function solely as an untouched final evaluation set. This introduces possible post hoc selection bias and means the small difference between XGBoost and Random Forest should be interpreted cautiously (Varma and Simon, 2006). Future work should define the selection rule in advance, choose the model using validation results only, freeze it, and then evaluate it once on the test set (Varma and Simon, 2006; Bischl et al., 2023).

6.6 Explainability Evaluation

SHAP was used on XGBoost, since it was the final model reported and needed more than just performance metrics to understand. The features were ranked by the mean absolute SHAP values that represent the average contribution of the feature across the observations (Lundberg and Lee, 2017). These values are relative influences, and do not on their own tell if a feature raises or lowers the likelihood of a subscription. Global importance outputs are included in Appendix Figures D.1 to D.4, and the numerical ranking in Appendix Table D.14.

6.7 Evaluation Limitations

Bank marketing dataset (PBM) is based on a historical Portuguese campaign and is not necessarily representative of other countries, periods or banking products (Moro, Cortez and Rita, 2014a). Despite the use of SMOTE and various evaluation measures, class imbalance was still an important challenge (He and Garcia, 2009; Fernández et al., 2018). There may be an artificial ordering of nominal categories using ordinal encoding. No external dataset was utilized for validating results and the artefact was not tested in a live bank-marketing environment. In addition, test performance is also used post hoc in final model selection, which also affects the independence of the final comparison reported.

6.8 Chapter Summary

Evaluation was done through various metrics, cross validation, hyper parameter optimisation, held-out assessment and SHAP interpretation. The findings from the exploratory, preprocessing, predictive and explainability are presented in Chapter 7 (Sokolova and Lapalme, 2009; Bischl et al., 2023; Lundberg and Lee, 2017).

CHAPTER 7: RESULTS

7.1 Introduction

The exploratory analysis, preprocessing, model-performance and SHAP results are presented in this chapter. The findings are reported without extended interpretation, which is reserved for Chapter 8. Detailed outputs are provided in Appendices B, C and D.

7.2 Exploratory Data Analysis Results

7.2.1 Dataset Structure

This dataset was originally collected from the Portuguese Bank Marketing dataset, which had 41.188 observations and 21 variables (Moro, Rita and Cortez, 2014b). There were 11 categorical and 10 numerical variables. Five integer and five floating point variables were present in the numeric group. The complete feature classification and descriptive statistics are provided in Appendix Tables B.1 and B.2.

Table 7.1: Dataset Structure and Feature Types

Characteristic	**Result**
Number of observations	41.188
Number of variables	21
Categorical features	11
Integer features	5
Floating-point features	5
Total numerical features	10

The average age of the customers was 40 years, ranging from 17 to 98 years old. The mean for call *"duration"* was 258.28 seconds, and maximum was 4.918 seconds. Average number of campaign "contacts" was 2.56 and the highest number of "contacts" made was 56. The economic

values were represented by the mean employment variation rate (0.08), mean “Euribor 3-month rate” (3.62), and mean number of “employees” (5.167.03).

7.2.2 Target Distribution

The target variable was highly skewed that consisted of (88.73%) 36.548 observations being non-subscribers, and a total of 4,640 observations (11.27%) made by subscribers. The whole target output is given in Appendix Table B.6.

Table 7.2: Target Variable Distribution

Subscription outcome	Count	Percentage
No	36.548	88.73%
Yes	4.640	11.27%
Total	41.188	100.00%

The first quality control check identified 12 duplicate rows, and no missing standards. Additionally, there were 6 categorical variables with the unknown category. The most common was default (8.597 observations) followed by education (1.731 observations). Complete results can be found in Appendix Table B.7.

Table 7.3: Summary of Data Quality Findings

Data-quality measure	Result
Duplicate rows	12
Missing values	0
Variables containing unknown	6
Highest unknown frequency	default: 8.597

7.2.3 Distributions and Relationships

Duration, campaign and previous had a positive skew on the histograms and boxplots. The clearest potential outliers occurred in age, duration, campaign and previous. The full distributions and boxplots are presented in Appendix Figures B.8 – B.8.3.

The highest positive numerical correlations were for emp.var.rate and euribor3m (0.97), euribor3m and nr.employed (0.95) and emp.var.rate and nr.employed (0.91). The highest negative correlation was between pdays and previous with -0.59. The entire correlation heatmap is included in Appendix Figure B.9.

There was also a category variable in terms of the different rates of subscriptions. The students' rate was 31.43% while that of blue-collar customers was 6.89%. The cell contact resulted in a 14.74% subscription rate, while the telephone contact resulted in a 5.23% subscription rate. Those customers that had a previous successful campaign had the highest subscription rate of all, at 65.11%, with 14.23% who had a previous failed campaign and 8.83% who didn't have a previous campaign. The full range of numerical and categorical comparisons are provided as Appendix Figure B.10 and Appendix Table B.11.

7.3 Preprocessing Results

After eliminating 12 duplicate observations, a total of 41.176 observations were available. The time variable (duration) was omitted and instead the binary variable previously_contacted was used for pdays (Moro, Rita and Cortez, 2014b). This variable could be used to select 1.515 customers already contacted and 39.661 customers not previously contacted. With target encoding, 36.537 non-subscriber records were generated as well as 4.639 subscriber records.

After dropping variables from the final data, 19 predictors remained. This split on the stratified data generated 24.705 training observations, 8.235 validation observations and 8.236 test observations.

The Logistic Regression preprocessing resulted in 52 encoded features and in the Random Forest and XGBoost branch, there were 19 encoded features. There were no missing values in either branch. Full results of the preprocessing steps are included in Appendix Tables C.1–C.13.

7.4 Model Results

7.4.1 Logistic Regression

The optimised Logistic Regression model got validated with an accuracy of 0.8282, precision of 0.3529, recall of 0.6282, F1-score of 0.4519 and ROC-AUC of 0.8003. On the test set, its accuracy was 0.8298, precision was 0.3589, recall was 0.6487, F1-score was 0.4621 and ROC-AUC was 0.7964. The complete outputs are given in Appendix Tables D.1 to D.4.

7.4.2 Random Forest

The optimised Random Forest has a validation accuracy of 0.8769, precision of 0.4608, recall of 0.5387, F1-score of 0.4967 and ROC-AUC of 0.7991. Its test accuracy was 0.8797, precision was 0.4707, recall was 0.5377, F1-score was 0.5020 and ROC-AUC was 0.7976. Full results are included in Appendix tables D.5-D.8.

7.4.3 XGBoost

The optimized XGBoost model validated the accuracy (0.8763), precision (0.4585), recall (0.5366), F1-score (0.4945) and ROC-AUC (0.7948). Its test accuracy was 0.8817, precision was 0.4782, recall was 0.5441, F1-score was 0.5090 and ROC-AUC was 0.7966. The outputs are detailed in the Appendix Tables D.9 – D.12.

7.4.4 Cross-Model Comparison

XGBoost produced the highest test F1-score at 0.5091, followed by Random Forest at 0.5020 and Logistic Regression at 0.4622. Random Forest recorded the highest test ROC-AUC at 0.7977, compared with 0.7967 for XGBoost and 0.7965 for Logistic Regression. Logistic Regression recorded the highest test recall at 0.6487. The full comparison and validation-to-test gaps are presented in Appendix Table D.13.

7.5 SHAP Results

The mean absolute SHAP ranking indicated that *"nr.employed"* is the most important variable with a value of 0.4471 followed by *"contact"* with 0.2290, *"emp.var.rate"* with 0.1543 and

"poutcome" with 0.1478. The values are the average magnitude of each feature's contribution and not the positive or negative direction of each feature's contribution (Lundberg and Lee, 2017). The full ranking can be found in Appendix Figure D.1 and Appendix Table D.14. Appendix Figures D.2, D.3 and D.4 provide additional model explanations.

7.6 Chapter Summary

The results showed significant class imbalance, important campaign and economic relations and similar performance of models. XGBoost gained the best Test F1-score and SHAP indicated that employment conditions, contact method and outcome of previous campaign were important features (Xie et al., 2023; Mishra et al., 2026).

CHAPTER 8: DISCUSSION

8.1 Introduction

This chapter interprets the findings presented in Chapter 7 in relation to the four research questions and previous bank-marketing research. The discussion connects the influential customer, financial, campaign and macroeconomic factors with the performance of Logistic Regression, Random Forest and XGBoost. It then considers how the results can support marketing targeting, together with the ethical and methodological limitations of the study.

8.2 RQ1: Influential Customer, Financial and Campaign Factors

RQ1 asked which customer, financial and campaign-related factors had the greatest influence on term-deposit subscription. SHAP identified *"nr.employed", "contact", "emp.var.rate"* and *"poutcome"* as the four leading predictors. The result shows that subscription predictions were shaped mainly by employment conditions, the contact channel and the outcome of previous campaigns rather than by demographic information alone (Moro, Cortez and Rita, 2014a; Xie et al., 2023).

"Contact" method was the second most influential feature. The exploratory analysis recorded a 14.74% subscription rate for cellular contacts compared with 5.23% for telephone contacts. This association may reflect differences in customer availability, campaign timing or communication practices, so it does not prove that changing the contact channel alone would increase subscriptions. Previous campaign *"poutcome"* was the fourth-ranked feature. Subscription reached 65.11% after a successful previous campaign, compared with 14.23% after a failed campaign and 8.83% where no previous campaign existed. This supports the use of campaign history when prioritising prospects (Moro, Cortez and Rita, 2014a; Apampa, 2016; Xie et al., 2023).

8.3 RQ2: Predicting Customer Subscription with Machine Learning

RQ2 asked whether machine-learning classifiers could predict subscription behaviour. All three models produced ROC-AUC values close to 0.80 on the test data, indicating useful but moderate separation between subscribers and non-subscribers. Their test F1-scores ranged from 0.4622 for

Logistic Regression to 0.5091 for XGBoost. These results show that the models learned meaningful patterns beyond the majority-class baseline, although none achieved sufficiently strong performance to justify automatic operational decisions.

Logistic Regression achieved the highest recall of 0.6487, meaning that it identified the largest proportion of actual subscribers, but its lower precision produced more false-positive contacts. Random Forest and XGBoost obtained better F1-scores because they achieved a stronger balance between precision and recall. Therefore, machine learning can support subscription prediction, but predictions should remain decision-support evidence and should be validated on current bank data before deployment (He and Garcia, 2009; Wainer, 2024).

The positive class represented only about 11.27% of observations. Accuracy was therefore insufficient on its own because a model could classify most non-subscribers correctly while missing many potential subscribers. The combined use of precision, recall, F1-score and ROC-AUC provided a more credible assessment of predictive usefulness (Sokolova and Lapalme, 2009; Fernández et al., 2018).

The findings are limited to this historical Portuguese campaign. Predictive ability in another country, period or product cannot be assumed without external validation.

8.4 RQ3: Comparative Model Performance

RQ3 asked which classifier demonstrated the strongest predictive performance. XGBoost achieved the highest test F1-score of 0.5091 and the highest combined F1-score and ROC-AUC ranking. Random Forest recorded a slightly higher test ROC-AUC of 0.7977, compared with 0.7967 for XGBoost, while Logistic Regression achieved the highest recall. Because F1-score was prioritised for the imbalanced outcome, XGBoost was reported as the strongest overall model, although the difference from Random Forest was small.

The ensemble models can represent nonlinear relationships and interactions that a standard Logistic Regression model may not capture. Random Forest reduces the instability of individual decision trees by aggregating many trees, while XGBoost builds trees sequentially and regularises model complexity

(Breiman, 2001; Chen and Guestrin, 2016). These properties help explain their stronger balance between precision and recall.

The selection must nevertheless be interpreted cautiously. The final comparison considered test performance, which introduces possible post hoc model-selection bias. A stronger future design would select and freeze the model using validation data alone before evaluating it once on the test set (Varma and Simon, 2006; Bischl et al., 2023).

8.5 RQ4: Can Machine Learning Predictive Insights Support More Effective Marketing Targeting.

RQ4 asked whether machine-learning insights could support more effective targeting. The findings provide practical decision-support evidence. Customers with successful previous campaign outcomes can be prioritised, contact method can inform campaign planning, and model scores can rank prospects rather than treating all customers as equally likely to subscribe. These measures may reduce unsuccessful contacts and focus staff time on customers with stronger predicted interest. Macroeconomic conditions also matter. The high SHAP importance of *"nr.employed"* and *"emp.var.rate"* indicates that campaign timing should consider the wider economic environment. However, strong correlations among *"emp.var.rate", "euribor3m"* and *"nr.employed"* mean that their individual effects should not be interpreted as independent causes. Campaign teams should also limit repeated contacts because subscribers generally received fewer contacts, although this observational analysis cannot determine an optimal contact limit (Moro, Cortez and Rita, 2014a; Ładyżyński, Żbikowski and Gawrysiak, 2019; Xie et al., 2023).

Model scores should support rather than replace professional judgement. Marketing staff should combine predictions with customer preferences, consent, campaign costs and organisational policy. The results answer RQ4 at a decision-support level, but the claimed efficiency gains remain provisional because no live campaign was conducted (Lundberg and Lee, 2017; Herhausen et al., 2024).

8.6 Ethical and Practical Implications

SHAP improves transparency by showing which variables influenced predictions, but explainability does not by itself establish fairness. Demographic and financial variables may reflect social inequalities or operate as proxies for protected characteristics. A bank should assess error rates across customer groups, minimise the data used and maintain meaningful human oversight.

If the model were applied to live customer data, the bank would also require a legitimate purpose, secure data handling and clear communication with customers. Predictions should not automatically exclude customers from receiving an offer.

8.7 Limitations

The dataset represents Portuguese campaigns conducted in an earlier economic period, so the findings may not transfer directly to current markets or other countries. No external dataset or live campaign was used to test generalisability, operational costs or customer responses. Ordinal encoding may have imposed artificial order on nominal categories, while correlated macroeconomic predictors complicate individual SHAP interpretations. Finally, the use of test comparisons in the reported model choice creates possible post hoc selection bias.

8.8 Chapter Summary

RQ1 identified employment conditions, contact method and previous campaign outcome as leading influences. RQ2 established that all three classifiers offered useful but moderate predictive ability. RQ3 found XGBoost to have the strongest reported balanced performance, although Random Forest achieved a marginally higher ROC-AUC. RQ4 showed that these predictions and explanations can support prospect prioritisation and campaign planning, but only with current-data validation, fairness checks and human oversight.

CHAPTER 9: CONCLUSION

9.1 Project Summary

This project explored the factors associated with customer interest in subscribing to a term deposit and how machine learning can be used to help target more customers more effectively for banks. This issue was tackled by the Portuguese Bank Marketing dataset, which was made up of customer, campaign and macro-economic data, but had a positive subscription rate of just 11.27% (Moro, Rita and Cortez, 2014b).

The methodological structure of business understanding, data exploration, data preparation, modelling, model evaluation and academic deployment were provided by CRISP-DM (Moro, Laureano and Cortez, 2011). The final Python Jupyter Notebook removed duplicate observations, dropped the leakage variable (duration) and created the stratified training, validation and test sets from the cleaned data (Moro, Cortez and Rita, 2014a). Logistic Regression, Random Forest and XGBoost classifiers were developed on algorithm specific pipelines (Mishra et al., 2026). They evaluated their performance on accuracy, precision, recall, F1/ROC-AUC prior to the use of the reported final model with SHAP (Sokolova and Lapalme, 2009; Lundberg and Lee, 2017).

9.2 Principal Conclusions

Results show that the purchase of term-deposits was not only related to demographic data but to a cocktail of campaign and macro-economic factors. Previous campaign outcome and method of contact were significant campaign related predictors and employment conditions made a significant contribution to the model's prediction. This is in line with previous studies that indicate bank-marketing results are influenced by the interplay between customer data and campaign design and economic conditions (Moro, Cortez and Rita, 2014a; Xie et al., 2023).

Logistic Regression was outperformed by Random Forest and XGBoost for the test dataset's F1-scores, which was a good sign that they performed well in terms of precision and recall. Logistic Regression revealed that it was able to predict a higher percentage of actual subscribers, but also made more false positive predictions. The optimal test F1-score reported the XGBoost with 0.5091 and

Random Forest achieved a marginally higher ROC-AUC of 0.7977 (Mishra et al., 2026; Xie et al., 2023).

The SHAP analysis identified *"nr.employed", "contact", "emp.var.rate"* and *"poutcome"* as the four leading predictors. These results highlight the need for an interpretation of model predictions rather than just reporting predictive scores (Lundberg and Lee, 2017).

9.3 Project Contribution and Achievement of Objectives

Objective 1 was fully achieved through the literature review, which established the academic context for bank-marketing prediction, imbalanced classification and explainable artificial intelligence. Objective 2 was fully achieved through exploratory data analysis of the 41.188-record dataset, including its structure, class imbalance, data-quality issues and relationships between subscription and customer, campaign and economic variables. Objective 3 was fully achieved through leakage-controlled preprocessing: 12 duplicate records and the duration variable were removed, pdays was converted into previously_contacted, the target was encoded, and algorithm-specific encoding, scaling, outlier capping and training-fold SMOTE procedures were implemented.
Objective 4 was fully achieved by developing and comparing Logistic Regression, Random Forest and XGBoost. Objective 5 was fully achieved through five-fold cross-validation, GridSearchCV and held-out accuracy, precision, recall, F1-score and ROC-AUC results. Objective 6 was substantially achieved. SHAP ranked *"nr.employed", "contact", "emp.var.rate" and "poutcome"* as the leading predictors, and these findings were translated into recommendations on prospect prioritisation, contact method, campaign timing and repeated contacts. This final objective was not fully demonstrated operationally because the recommendations were not tested in a live bank campaign. Overall, the project contributed a reproducible, leakage-controlled and explainable decision-support process, while recognising that external validation is required before deployment.

9.4 Recommendations

Bank marketers should focus on using model scores to priorities customers, and not treat them all the same. When planning campaigns, it is essential to take previous campaign successes, contact

channel and economic conditions into consideration. The number of contacts should be minimised as much as possible to minimise cost and inconvenience to customers. Predictions should augment human decision making and should not be uniquely used to decide whether to present a customer an offer.

9.5 Future Work

Further studies are needed to test the models with more up-to-date data for various countries, institutions and financial products. Other forms of categorical encoding should be tried as a method to prevent arbitrary ordering of nominal categories (Pedregosa et al., 2011). There may be ways to optimise thresholds and costs to balance subscribers missed and contacts also sent, with threshold optimisation and cost-sensitive learning being two possibilities (He and Garcia, 2009; Fernández et al., 2018).

The modelling process should also be used to determine the selection rule prior to considering test performance (Varma and Simon, 2006). Then the model selected from the validation result should be frozen and tested once with the test set (Varma and Simon, 2006; Bischl et al., 2023). Live operational testing may be able to measure campaign costs, customer responses and actual usability. Fairness evaluations should consider if there are variations in prediction errors between demographic or financial groups.

9.6 Final Closing Statement

With careful validation and understandable explanations and responsible human decision-making, machine learning can assist with more focused bank marketing when coupled with their predictive performance.

REFERENCE LIST

Agarwal, A., Prabha, S. and Yadav, R. (2025) 'Exploratory Data Analysis (EDA) for Banking and Finance: Unveiling Insights and Patterns', GBS Impact: Journal of Multi Disciplinary Research, 11(1), pp. 249-280. Available at: https://doi.org/10.58419/gbs.v11i1.1112520.

Apampa, O. (2016) 'Evaluation of classification and ensemble algorithms for bank customer marketing response prediction', *Journal of International Technology and Information Management*, 25(4), pp. 85-100. Available at: https://doi.org/10.58729/1941-6679.1296.

Belkin, M., Hsu, D., Ma, S. and Mandal, S. (2019) 'Reconciling modern machine-learning practice and the classical bias-variance trade-off', *Proceedings of the National Academy of Sciences*, 116(32), pp. 15849-15854. Available at: https://doi.org/10.1073/pnas.1903070116.

Bischl, B., Binder, M., Lang, M., Pielok, T., Richter, J., Coors, S., Thomas, J., Ullmann, T., Becker, M., Boulesteix, A.-L., Deng, D. and Lindauer, M. (2023) 'Hyperparameter optimization: Foundations, algorithms, best practices and open challenges', *WIREs Data Mining and Knowledge Discovery*, 13(2), e1484. Available at: https://doi.org/10.1002/widm.1484.

Boukerche, A., Zheng, L. and Alfandi, O. (2020) 'Outlier detection: Methods, models, and classification', *ACM Computing Surveys*, 53(3), Article 55, pp. 1-37. Available at: https://doi.org/10.1145/3381028.

Breiman, L. (2001) 'Random forests', *Machine Learning*, 45(1), pp. 5-32. Available at: https://doi.org/10.1023/A:1010933404324.

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C. and Wirth, R. (2000) *CRISP-DM 1.0: Step-by-step data mining guide*. Chicago, IL: SPSS Inc.

Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002) 'SMOTE: Synthetic minority over-sampling technique', *Journal of Artificial Intelligence Research*, 16, pp. 321-357. Available at: https://doi.org/10.1613/jair.953.

Chen, T. and Guestrin, C. (2016) 'XGBoost: A scalable tree boosting system', in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New

York: Association for Computing Machinery, pp. 785-794. Available at: https://doi.org/10.1145/2939672.2939785.

Chu, X., Ilyas, I.F., Krishnan, S. and Wang, J. (2016) 'Data cleaning: Overview and emerging challenges', in *Proceedings of the 2016 International Conference on Management of Data*. New York: Association for Computing Machinery, pp. 2201-2206. Available at: https://doi.org/10.1145/2882903.2912574.

Data Protection Act 2018, c. 12. Available at: https://www.legislation.gov.uk/ukpga/2018/12/contents (Accessed: 11 August 2026).

Fernández, A., García, S., Herrera, F. and Chawla, N.V. (2018) 'SMOTE for learning from imbalanced data: Progress and challenges, marking the 15-year anniversary', *Journal of Artificial Intelligence Research*, 61, pp. 863-905. Available at: https://doi.org/10.1613/jair.1.11192.

Ganar, C. and Hosein, P. (2022) 'Customer segmentation for improving marketing campaigns in the banking industry', in *2022 5th Asia Conference on Machine Learning and Computing*. Piscataway, NJ: IEEE, pp. 48-52. Available at: https://doi.org/10.1109/ACMLC58173.2022.00017.

Harris, C.R., Millman, K.J., van der Walt, S.J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N.J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M.H., Brett, M., Haldane, A., del Río, J.F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C. and Oliphant, T.E. (2020) 'Array programming with NumPy', Nature, 585, pp. 357-362. Available at: https://doi.org/10.1038/s41586-020-2649-2.

Harris, J.K. (2021) 'Primer on binary logistic regression', *Family Medicine and Community Health*, 9(Suppl 1), e001290. Available at: https://doi.org/10.1136/fmch-2021-001290.

He, H. and Garcia, E.A. (2009) 'Learning from imbalanced data', *IEEE Transactions on Knowledge and Data Engineering*, 21(9), pp. 1263-1284. Available at: https://doi.org/10.1109/TKDE.2008.239.

Herhausen, D., Bernritter, S.F., Ngai, E.W.T., Kumar, A. and Delen, D. (2024) 'Machine learning in marketing: Recent progress and future research directions', *Journal of Business Research*, 170, Article 114254. Available at: https://doi.org/10.1016/j.jbusres.2023.114254.

Hunter, J.D. (2007) 'Matplotlib: A 2D graphics environment', Computing in Science and Engineering, 9(3), pp. 90-95. Available at: https://doi.org/10.1109/MCSE.2007.55.

Kaufman, S., Rosset, S., Perlich, C. and Stitelman, O. (2012) 'Leakage in data mining: Formulation, detection, and avoidance', *ACM Transactions on Knowledge Discovery from Data*, 6(4), Article 15, pp. 1-21. Available at: https://doi.org/10.1145/2382577.2382579.

Keshava, M.C., Chandana, K.S., Sagar, K.S. and Harish, M.N. (2019) 'Predictive analysis on bank marketing campaign using machine learning algorithms', *International Journal for Research in Applied Science and Engineering Technology*, 7(4), pp. 2664-2668. Available at: https://doi.org/10.22214/ijraset.2019.4483.

Lemaître, G., Nogueira, F. and Aridas, C.K. (2017) 'Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning', *Journal of Machine Learning Research*, 18(17), pp. 1-5. Available at: https://jmlr.org/papers/v18/16-365.html.

Lundberg, S.M. and Lee, S.-I. (2017) 'A unified approach to interpreting model predictions', in Guyon, I., von Luxburg, U., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S. and Garnett, R. (eds.) *Advances in Neural Information Processing Systems 30*. Red Hook, NY: Curran Associates, pp. 4765-4774. Available at: https://proceedings.neurips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions.

Ładyżyński, P., Żbikowski, K. and Gawrysiak, P. (2019) 'Direct marketing campaigns in retail banking with the use of deep learning and random forests', *Expert Systems with Applications*, 134, pp. 28-35. Available at: https://doi.org/10.1016/j.eswa.2019.05.020.

McKinney, W. (2010) 'Data structures for statistical computing in Python', in van der Walt, S. and Millman, J. (eds.) Proceedings of the 9th Python in Science Conference. Austin, TX: SciPy, pp. 56-61. Available at: https://doi.org/10.25080/Majora-92bf1922-00a.

Mishra, H., Yadav, S., Agrawal, S., Hazra, D., Gupta, N. and Raj, M. (2026) 'Bank Marketing Campaign Response Prediction in Digital ERA', in Proceedings of the International Conference on Sustainable Computing and Artificial Intelligence (ICSCAI 2025). Advances in Engineering Research. Paris: Atlantis Press, pp. 377-390. Available at: https://doi.org/10.2991/978-94-6239-674-6_32.

Moro, S., Laureano, R.M.S. and Cortez, P. (2011) 'Using data mining for bank direct marketing: An application of the CRISP-DM methodology', in Novais, P., Machado, J., Analide, C. and Abelha, A.

(eds.) *Proceedings of the European Simulation and Modelling Conference*. Guimarães: EUROSIS, pp. 117-121.

Moro, S., Cortez, P. and Rita, P. (2014a) 'A data-driven approach to predict the success of bank telemarketing', *Decision Support Systems*, 62, pp. 22-31. Available at: https://doi.org/10.1016/j.dss.2014.03.001.

Moro, S., Rita, P. and Cortez, P. (2014b) *Bank Marketing* [dataset]. Irvine, CA: UCI Machine Learning Repository. Available at: https://doi.org/10.24432/C5K306.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., VanderPlas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825-2830. Available at: https://jmlr.org/papers/v12/pedregosa11a.html.

Polyzotis, N., Zinkevich, M., Roy, S., Breck, E. and Whang, S. (2019) 'Data validation for machine learning', *Proceedings of Machine Learning and Systems*, 1, pp. 334-347. Available at: https://proceedings.mlsys.org/paper_files/paper/2019/hash/928f1160e52192e3e0017fb63ab65391-Abstract.html.

Sokolova, M. and Lapalme, G. (2009) 'A systematic analysis of performance measures for classification tasks', *Information Processing and Management*, 45(4), pp. 427-437. Available at: https://doi.org/10.1016/j.ipm.2009.03.002.

Varma, S. and Simon, R. (2006) 'Bias in error estimation when using cross-validation for model selection', *BMC Bioinformatics*, 7, Article 91. Available at: https://doi.org/10.1186/1471-2105-7-91.

Wainer, J. (2024) 'An empirical evaluation of imbalanced data strategies from a practitioner's point of view', *Expert Systems with Applications*, 256, Article 124863. Available at: https://doi.org/10.1016/j.eswa.2024.124863.

Waskom, M.L. (2021) 'seaborn: Statistical data visualization', Journal of Open Source Software, 6(60), Article 3021. Available at: https://doi.org/10.21105/joss.03021.

Xie, C., Zhang, J.-L., Zhu, Y., Xiong, B. and Wang, G.-J. (2023) 'How to improve the success of bank telemarketing? Prediction and interpretability analysis based on machine learning', *Computers*

and Industrial Engineering, 175, Article 108874. Available at:

https://doi.org/10.1016/j.cie.2022.108874

APPENDIX A: SOFTWARE ARTEFACT IMPLEMENTED

This appendix presents the Python code implemented for the bank marketing classification pipeline, referenced throughout Chapter 3 (Methodology). Code is organised by pipeline stage, with each stage presented as a numbered sub-section.

A1 Data import

Listing A1.1: Library imports and environment setup

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import collections
import seaborn as sns
import matplotlib
import sklearn
from sklearn.model_selection import train_test_split
import imblearn
import xgboost
import shap

!pip install imbalanced-learn
!pip install xgboost
!pip install shap
```

Listing A1.2: Dataset loading

```
FILE_PATH = "data/bank-additional-full.csv"

# Load CSV file into a pandas DataFrame
# sep=";" — this file uses semicolons instead of commas
df = pd.read_csv(FILE_PATH, sep=";")
```

Listing A1.3: Data inspection

```
# Shape of the dataset
print("Shape:", df.shape)

# Inspect data types
print("\nData types:")
print(df.dtypes)

# Confirm target variable 'y' exists
print("\nIs 'y' in the columns?:", "y" in df.columns)

# Confirm target classes and their counts
print("\nTarget class counts:")
print(df["y"].value_counts())

# Preview the first 20 rows
df.head(20)
```

A2 Exploratory data analysis

Listing A2.1: Dataset structure and feature types

```
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
print("Feature type counts:")
```

```
print(df.dtypes.value_counts())

numerical_cols = df.select_dtypes(include=["int64", "float64"]).columns.tolist()
categorical_cols = df.select_dtypes(include=["object"]).columns.tolist()

print("Numerical features:", numerical_cols)
print("Categorical features:", categorical_cols)
```

Listing A2.2: Summary statistics and categorical values

```
print(df[numerical_cols].describe())

for col in categorical_cols:
    print(col)
    print(df[col].value_counts())
```

Listing A2.3: Target distribution

```
print(df["y"].value_counts())
print((df["y"].value_counts(normalize=True) * 100).round(2))

plt.figure(figsize=(6, 4))
sns.countplot(data=df, x="y", hue="y", legend=False)
plt.title("How is the target variable distributed?")
plt.xlabel("Subscribed (y)")
plt.ylabel("Count")
plt.show()
```

Listing A2.4: Data quality checks

```
print("Number of duplicate rows:", df.duplicated().sum())

print("\nMissing values per column:")
print(df.isnull().sum())

print("\n'unknown' values per column:")
unknown_counts = (df == "unknown").sum()
print(unknown_counts[unknown_counts > 0])
```

Listing A2.5: Numerical distributions and outliers

```
df[numerical_cols].hist(bins=30, figsize=(15, 10))
plt.suptitle("What do the numerical feature distributions look like?")
plt.tight_layout()
plt.show()

n_cols = 3
n_rows = (len(numerical_cols) + n_cols - 1) // n_cols
fig, axes = plt.subplots(n_rows, n_cols, figsize=(15, 4 * n_rows))
axes = axes.flatten()

for i, col in enumerate(numerical_cols):
    sns.boxplot(data=df, y=col, ax=axes[i])
    axes[i].set_title(col)

for j in range(i + 1, len(axes)):
    axes[j].axis("off")

plt.suptitle("Which numerical features contain outliers?")
plt.tight_layout()
```

```
plt.show()
```

Listing A2.6: Correlations between numerical features

```
plt.figure(figsize=(10, 8))
sns.heatmap(df[numerical_cols].corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("How do numerical features correlate with each other?")
plt.show()
```

Listing A2.7: Feature relationships with subscription outcome

```
# Numerical features by subscription outcome
n_cols = 3
n_rows = (len(numerical_cols) + n_cols - 1) // n_cols
fig, axes = plt.subplots(n_rows, n_cols, figsize=(15, 4 * n_rows))
axes = axes.flatten()

for i, col in enumerate(numerical_cols):
    sns.boxplot(data=df, x="y", y=col, hue="y", legend=False, ax=axes[i])
    axes[i].set_title(col)

for j in range(i + 1, len(axes)):
    axes[j].axis("off")

plt.suptitle("How do numerical features differ by subscription outcome?")
plt.tight_layout()
plt.show()

# Categorical features — subscription rate per category
for col in categorical_cols:
    if col == "y":
        continue
    rate = df.groupby(col)["y"].apply(lambda x: (x == "yes").mean() * 100).sort_values(ascending=False)
    print("Subscription rate (%) by", col)
    print(rate.round(2))
```

A3 Data preprocessing

Listing A3.1: Copy of original dataset

```
df_raw = df.copy()
df_clean = df.copy()
```

Listing A3.2: Remove duplicates

```
rows_before = df_clean.shape[0]
df_clean = df_clean.drop_duplicates()
df_clean = df_clean.reset_index(drop=True)
rows_after = df_clean.shape[0]

print("Rows before:", rows_before)
print("Rows after:", rows_after)
print("Duplicates removed:", rows_before - rows_after)
```

Listing A3.3: Drop duration (data leakage)

```
# duration is only known after a call has ended
# including it causes data leakage and inflates model performance
if 'duration' in df_clean.columns:
    df_clean = df_clean.drop(columns=['duration'])

print("Shape after removal:", df_clean.shape)
```

```
print("'duration' in columns:", 'duration' in df_clean.columns)
```

Listing A3.4: Feature engineering: pdays → previously_contacted

```
df_clean['previously_contacted'] = (df_clean['pdays'] != 999).astype(int)
df_clean = df_clean.drop(columns=['pdays'])

print(df_clean['previously_contacted'].value_counts())
print("'pdays' in columns:", 'pdays' in df_clean.columns)
```

Listing A3.5: Target variable encoding

```
print("Before encoding:")
print(df_clean['y'].value_counts())

df_clean['y'] = df_clean['y'].map({'yes': 1, 'no': 0}).astype(int)

print("\nAfter encoding:")
print(df_clean['y'].value_counts())

print("\nClass proportions:")
print(df_clean['y'].value_counts(normalize=True))

print("\nMissing values in y:", df_clean['y'].isnull().sum())
```

Listing A3.6: Separate predictors and target

```
X = df_clean.drop(columns=['y'])
y = df_clean['y']

print("X shape:", X.shape)
print("y shape:", y.shape)
print("\nPredictor columns:", list(X.columns))

dummy_check = [col for col in X.columns if '_' in col and X[col].nunique() == 2]
print("\nPossible existing dummy variables:", dummy_check)
```

Listing A3.7: Train-test-validation split (60/20/20, stratified)

```
from sklearn.model_selection import train_test_split

X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)
X_train, X_validation, y_train, y_validation = train_test_split(
    X_temp, y_temp, test_size=0.25, stratify=y_temp, random_state=42
)

X_train = X_train.copy()
X_validation = X_validation.copy()
X_test = X_test.copy()

print("X_train:", X_train.shape)
print("X_validation:", X_validation.shape)
print("X_test:", X_test.shape)
print("y_train:", y_train.shape)
print("y_validation:", y_validation.shape)
print("y_test:", y_test.shape)
```

Listing A3.8: Validate train-test-validation split

```
total = len(X)
train_pct = len(X_train) / total
val_pct = len(X_validation) / total
test_pct = len(X_test) / total

print("Training proportion:", round(train_pct, 3))
print("Validation proportion:", round(val_pct, 3))
print("Test proportion:", round(test_pct, 3))
print("Total proportion:", round(train_pct + val_pct + test_pct, 3))

print("\nClass balance (train):", y_train.mean())
print("Class balance (validation):", y_validation.mean())
print("Class balance (test):", y_test.mean())

train_idx = set(X_train.index)
val_idx = set(X_validation.index)
test_idx = set(X_test.index)

assert train_idx.isdisjoint(val_idx)
assert train_idx.isdisjoint(test_idx)
assert val_idx.isdisjoint(test_idx)
assert len(X_train) + len(X_validation) + len(X_test) == len(X)

print("\nAll checks passed: no overlap, all records accounted for.")
```

Listing A3.9: Identify feature types

```
numerical_features = X_train.select_dtypes(include=['int64', 'float64']).columns.tolist()
categorical_features = X_train.select_dtypes(include=['object']).columns.tolist()

print("Numerical features:", numerical_features)
print("Number of numerical features:", len(numerical_features))
print("\nCategorical features:", categorical_features)
print("Number of categorical features:", len(categorical_features))

assert len(categorical_features) > 0, "No categorical features found"
```

Listing A3.10: Outlier capping (IQR method — fit on train, applied to train and test)

```
outlier_bounds = {}
for col in ['age', 'campaign']:
    Q1 = X_train[col].quantile(0.25)
    Q3 = X_train[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outlier_bounds[col] = (lower, upper)

print("Learned bounds:", outlier_bounds)

for col, (lower, upper) in outlier_bounds.items():
    X_train[col] = X_train[col].clip(lower, upper)
    X_validation[col] = X_validation[col].clip(lower, upper)
    X_test[col] = X_test[col].clip(lower, upper)

print("\nSummary after clipping (train):")
print(X_train[['age', 'campaign']].describe())
```

Listing A3.11: One-hot encoding (Branch A — logistic regression)

```
from sklearn.preprocessing import OneHotEncoder
```

```
encoder = OneHotEncoder(drop='first', handle_unknown='ignore', sparse_output=False)
encoder.fit(X_train[categorical_features])
encoded_cols = encoder.get_feature_names_out(categorical_features)

X_train_cat = pd.DataFrame(encoder.transform(X_train[categorical_features]), columns=encoded_cols,
index=X_train.index)
X_validation_cat = pd.DataFrame(encoder.transform(X_validation[categorical_features]), columns=encoded_cols,
index=X_validation.index)
X_test_cat = pd.DataFrame(encoder.transform(X_test[categorical_features]), columns=encoded_cols, index=X_test.index)

X_train_processed = pd.concat([X_train[numerical_features], X_train_cat], axis=1)
X_validation_processed = pd.concat([X_validation[numerical_features], X_validation_cat], axis=1)
X_test_processed = pd.concat([X_test[numerical_features], X_test_cat], axis=1)

print("X_train_processed:", X_train_processed.shape)
print("X_validation_processed:", X_validation_processed.shape)
print("X_test_processed:", X_test_processed.shape)
print("\nExample encoded feature names:", list(encoded_cols[:5]))

assert X_train_processed.isnull().sum().sum() == 0
assert list(X_train_processed.columns) == list(X_validation_processed.columns) == list(X_test_processed.columns)
assert X_train_processed.shape[0] == y_train.shape[0]
print("\nAll checks passed.")
```

Listing A3.12: Feature scaling (Branch A — logistic regression)

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaler.fit(X_train_processed)

X_train_lr = pd.DataFrame(
    scaler.transform(X_train_processed),
    columns=X_train_processed.columns,
    index=X_train_processed.index
)
X_validation_lr = pd.DataFrame(
    scaler.transform(X_validation_processed),
    columns=X_validation_processed.columns,
    index=X_validation_processed.index
)
X_test_lr = pd.DataFrame(
    scaler.transform(X_test_processed),
    columns=X_test_processed.columns,
    index=X_test_processed.index
)

print("X_train_lr:", X_train_lr.shape)
print("X_validation_lr:", X_validation_lr.shape)
print("X_test_lr:", X_test_lr.shape)
```

Listing A3.13: Ordinal encoding (Branch B — Random Forest and XGBoost)

```
from sklearn.preprocessing import OrdinalEncoder

ordinal_encoder = OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1)
ordinal_encoder.fit(X_train[categorical_features])

X_train_cat_ord = pd.DataFrame(
    ordinal_encoder.transform(X_train[categorical_features]),
    columns=categorical_features,
```

```
    index=X_train.index
)
X_validation_cat_ord = pd.DataFrame(
    ordinal_encoder.transform(X_validation[categorical_features]),
    columns=categorical_features,
    index=X_validation.index
)
X_test_cat_ord = pd.DataFrame(
    ordinal_encoder.transform(X_test[categorical_features]),
    columns=categorical_features,
    index=X_test.index
)

X_train_tree = pd.concat([X_train[numerical_features], X_train_cat_ord], axis=1)
X_validation_tree = pd.concat([X_validation[numerical_features], X_validation_cat_ord], axis=1)
X_test_tree = pd.concat([X_test[numerical_features], X_test_cat_ord], axis=1)

print("X_train_tree:", X_train_tree.shape)
print("X_validation_tree:", X_validation_tree.shape)
print("X_test_tree:", X_test_tree.shape)
```

Listing A3.14: Final preprocessing validation

```
print("Missing values - LR branch:")
print(X_train_lr.isnull().sum().sum(), X_validation_lr.isnull().sum().sum(), X_test_lr.isnull().sum().sum())

print("\nMissing values - Tree branch:")
print(X_train_tree.isnull().sum().sum(), X_validation_tree.isnull().sum().sum(), X_test_tree.isnull().sum().sum())

print("\nLR columns match:", list(X_train_lr.columns) == list(X_validation_lr.columns) == list(X_test_lr.columns))
print("Tree columns match:", list(X_train_tree.columns) == list(X_validation_tree.columns) == list(X_test_tree.columns))

print("\nRow counts match y - LR:")
print(X_train_lr.shape[0] == y_train.shape[0])
print(X_validation_lr.shape[0] == y_validation.shape[0])
print(X_test_lr.shape[0] == y_test.shape[0])

print("\nRow counts match y - Tree:")
print(X_train_tree.shape[0] == y_train.shape[0])
print(X_validation_tree.shape[0] == y_validation.shape[0])
print(X_test_tree.shape[0] == y_test.shape[0])
```

A4 Logistic regression model

Listing A4.1: Logistic Regression 5-fold cross-validation

```
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_validate
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scoring = ['accuracy', 'precision', 'recall', 'f1', 'roc_auc']
lr_pipeline = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('classifier', LogisticRegression(random_state=42, max_iter=1000))
])
cv_results = cross_validate(lr_pipeline, X_train_lr, y_train, cv=cv, scoring=scoring)
print("LR (SMOTE pipeline) 5-fold CV")
for metric in scoring:
    scores = cv_results['test_' + metric]
    print(metric, ":", round(scores.mean(), 4), "+/-", round(scores.std(), 4))
```

Listing A4.2: GridSearch hyperparameter tuning

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'classifier__C': [0.001, 0.005, 0.01, 0.1, 1, 10],
    'classifier__penalty': ['l1', 'l2'],
    'classifier__solver': ['liblinear']
}

grid_search = GridSearchCV(lr_pipeline, param_grid, cv=cv, scoring='f1', n_jobs=-1)
grid_search.fit(X_train_lr, y_train)

print("Best parameters:", grid_search.best_params_)
print("Best CV F1:", grid_search.best_score_)

evaluate(grid_search.best_estimator_, X_validation_lr, y_validation, "Validation set (chosen model)")
```

Listing A4.3: Applying optimal parameters obtained from GridSearch

```
lr_final = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('classifier', LogisticRegression(
        C=10, penalty='l2', solver='liblinear',
        random_state=42, max_iter=1000
    ))
])
lr_final.fit(X_train_lr, y_train)

evaluate(lr_final, X_train_lr, y_train, "Training set (final model)")
evaluate(lr_final, X_validation_lr, y_validation, "Validation set (final model)")
```

Listing A4.4: Final evaluation — test set (LR frozen)

```
from sklearn.metrics import ConfusionMatrixDisplay, RocCurveDisplay
import matplotlib.pyplot as plt

evaluate(lr_final, X_test_lr, y_test, "Test set (final LR evaluation)")

# Confusion matrix
ConfusionMatrixDisplay.from_estimator(lr_final, X_test_lr, y_test, display_labels=['No','Yes'], cmap='Blues')
plt.title("Confusion Matrix - Logistic Regression (Test Set)")
plt.show()

# ROC-AUC curve
RocCurveDisplay.from_estimator(lr_final, X_test_lr, y_test)
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random guess')
plt.title("ROC Curve - Logistic Regression (Test Set)")
plt.legend()
plt.show()
```

A4.2.1 Random Forest algorithm

Listing A4.2.1: Random Forest 5-fold cross-validation

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold, cross_validate
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scoring = ['accuracy', 'precision', 'recall', 'f1', 'roc_auc']
rf_pipeline = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
```

```
    ('classifier', RandomForestClassifier(random_state=42))
])
cv_results = cross_validate(rf_pipeline, X_train_tree, y_train, cv=cv, scoring=scoring)
print("RF (SMOTE pipeline) 5-fold CV")
for metric in scoring:
    scores = cv_results['test_' + metric]
    print(metric, ":", round(scores.mean(), 4), "+/-", round(scores.std(), 4))
```

Listing A4.2.2: GridSearch hyperparameter tuning

```
param_grid_rf = {
    'classifier__n_estimators': [100, 200, 300],
    'classifier__max_depth': [None, 10, 20],
    'classifier__min_samples_split': [2, 5],
    'classifier__min_samples_leaf': [1, 2]
}

grid_search_rf = GridSearchCV(rf_pipeline, param_grid_rf, cv=cv, scoring='f1', n_jobs=-1)
grid_search_rf.fit(X_train_tree, y_train)

print("Best parameters:", grid_search_rf.best_params_)
print("Best CV F1:", grid_search_rf.best_score_)

evaluate(grid_search_rf.best_estimator_, X_validation_tree, y_validation, "Validation set (chosen model)")
```

Listing A4.2.3: Applying optimal parameters obtained from GridSearch

```
rf_final = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('classifier', RandomForestClassifier(
        n_estimators=200, max_depth=10,
        min_samples_split=5, min_samples_leaf=2,
        random_state=42
    ))
])
rf_final.fit(X_train_tree, y_train)

evaluate(rf_final, X_train_tree, y_train, "Training set (final model)")
evaluate(rf_final, X_validation_tree, y_validation, "Validation set (final model)")
```

Listing A4.2.4: Final evaluation — test set (once, RF frozen)

```
from sklearn.metrics import ConfusionMatrixDisplay, RocCurveDisplay
import matplotlib.pyplot as plt

evaluate(rf_final, X_test_tree, y_test, "Test set (final RF evaluation)")

# Confusion matrix
ConfusionMatrixDisplay.from_estimator(rf_final, X_test_tree, y_test, display_labels=['No','Yes'], cmap='Blues')
plt.title("Confusion Matrix - Random Forest (Test Set)")
plt.show()

# ROC-AUC curve
RocCurveDisplay.from_estimator(rf_final, X_test_tree, y_test)
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random guess')
plt.title("ROC Curve - Random Forest (Test Set)")
plt.legend()
plt.show()
```

A4.2.2 XGBoost algorithm

Listing A4.3.1: XGBoost 5-fold cross-validation

```
from xgboost import XGBClassifier
from sklearn.model_selection import StratifiedKFold, cross_validate
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scoring = ['accuracy', 'precision', 'recall', 'f1', 'roc_auc']
xgb_pipeline = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('classifier', XGBClassifier(random_state=42, eval_metric='logloss'))
])
cv_results = cross_validate(xgb_pipeline, X_train_tree, y_train, cv=cv, scoring=scoring)
print("XGBoost (SMOTE pipeline) 5-fold CV")
for metric in scoring:
    scores = cv_results['test_' + metric]
    print(metric, ":", round(scores.mean(), 4), "+/-", round(scores.std(), 4))
```

Listing A4.4.2: GridSearch hyperparameter tuning

```
param_grid_xgb = {
    'classifier__n_estimators': [100, 200, 300],
    'classifier__max_depth': [3, 5, 7],
    'classifier__learning_rate': [0.01, 0.1, 0.2],
    'classifier__subsample': [0.8, 1.0]
}

grid_search_xgb = GridSearchCV(xgb_pipeline, param_grid_xgb, cv=cv, scoring='f1', n_jobs=-1)
grid_search_xgb.fit(X_train_tree, y_train)

print("Best parameters:", grid_search_xgb.best_params_)
print("Best CV F1:", grid_search_xgb.best_score_)

evaluate(grid_search_xgb.best_estimator_, X_validation_tree, y_validation, "Validation set (chosen model)")
```

Listing A4.4.3: Final model — using GridSearchCV best parameters

```
xgb_final = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('classifier', XGBClassifier(
        n_estimators=200, max_depth=5,
        learning_rate=0.01, subsample=1.0,
        random_state=42, eval_metric='logloss'
    ))
])
xgb_final.fit(X_train_tree, y_train)

evaluate(xgb_final, X_train_tree, y_train, "Training set (final model)")
evaluate(xgb_final, X_validation_tree, y_validation, "Validation set (final model)")
```

Listing A4.4.4: Final evaluation — test set (once, XGBoost frozen)

```
from sklearn.metrics import ConfusionMatrixDisplay, RocCurveDisplay
import matplotlib.pyplot as plt

evaluate(xgb_final, X_test_tree, y_test, "Test set (final XGBoost evaluation)")

# Confusion matrix
ConfusionMatrixDisplay.from_estimator(xgb_final, X_test_tree, y_test, display_labels=['No','Yes'], cmap='Blues')
plt.title("Confusion Matrix - XGBoost (Test Set)")
plt.show()
```

```
# ROC-AUC curve
RocCurveDisplay.from_estimator(xgb_final, X_test_tree, y_test)
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random guess')
plt.title("ROC Curve - XGBoost (Test Set)")
plt.legend()
plt.show()
```

A5 Model comparison

Listing A5.1 — Scoring, ranking, and overfitting check across all three models

```
import pandas as pd

# Score all three models on one dataset (validation or test)
def score_models(X_lr, X_tree, y):
    return pd.DataFrame({
        'Logistic Regression': get_metrics(lr_final, X_lr, y),
        'Random Forest': get_metrics(rf_final, X_tree, y),
        'XGBoost': get_metrics(xgb_final, X_tree, y)
    }).T.round(4)

# Step 1: score on validation data (used to pick the model)
val_df = score_models(X_validation_lr, X_validation_tree, y_validation)

# Step 2: score on test data (final, unseen data)
test_df = score_models(X_test_lr, X_test_tree, y_test)

# Step 3: rank by the average of F1 and ROC-AUC (best two metrics for imbalanced data)
test_df['F1 + ROC-AUC (avg)'] = ((test_df['F1'] + test_df['ROC-AUC']) / 2).round(4)
test_df = test_df.sort_values('F1 + ROC-AUC (avg)', ascending=False)

# Step 4: check overfitting — how much each score drops from validation to test
gap_df = (val_df - test_df[val_df.columns]).round(4)

# Print tables
print("Table X.X — Validation Performance\n", val_df, "\n")
print("Table X.X — Test Performance (Final)\n", test_df, "\n")
print("Table X.X — Final Ranking (F1 + ROC-AUC average)\n", test_df[['F1', 'ROC-AUC', 'F1 + ROC-AUC (avg)']], "\n")
print("Table X.X — Validation–Test Gap\n", gap_df, "\n")
print("Final selected model:", test_df['F1 + ROC-AUC (avg)'].idxmax())
```

A6 SHAP analysis

Listing A6.1 — SHAP interpretability analysis on the final XGBoost model

```
import shap
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 1. Pull the raw XGBoost model out of the pipeline
xgb_model = xgb_final.named_steps['classifier']

# 2. TreeExplainer on the bare model
explainer = shap.TreeExplainer(xgb_model)

# 3. SHAP values on the test set
#    SMOTE only affects training, so it does NOT transform test data —
#    the test features go straight to the classifier unchanged.
shap_values = explainer.shap_values(X_test_tree)
```

```
# 4. Bar plot — overall importance
shap.summary_plot(shap_values, X_test_tree, plot_type="bar", show=False)
plt.title("Figure X.X — Feature Importance (mean |SHAP value|)")
plt.tight_layout()
plt.show()

# 5. Ranked table
importance = pd.DataFrame({
    'Feature': X_test_tree.columns,
    'Mean |SHAP|': np.abs(shap_values).mean(axis=0)
}).sort_values('Mean |SHAP|', ascending=False).reset_index(drop=True)
importance['Rank'] = importance.index + 1

print("Table X.X — SHAP Feature Importance Ranking\n")
print(importance.to_string(index=False))
```

APPENDIX B: COMPLETE EDA OUTPUTS

The appendix presents the complete exploratory data analysis outputs for the Bank Marketing dataset (Moro et al., 2014). Only the interpreted findings arising from this analysis are discussed in Chapter 6; the full outputs are provided here for verification and reproducibility.

Table B.0 – Dataset Structure and Feature Types

Output of Step 3.1 (EDA — Dataset Structure & Feature Types), confirming the raw dimensions of the Bank Marketing dataset (Moro et al., 2014) and the type split used to route features into the numerical and categorical preprocessing branches described in Appendix A, Section A3.

Property	Value
Rows	41,188
Columns	21
Feature type counts	object: 11 | int64: 5 | float64: 5
Numerical features (10)	age, duration, campaign, pdays, previous, emp.var.rate, cons.price.idx, cons.conf.idx, euribor3m, nr.employed
Categorical features (11)	job, marital, education, default, housing, loan, contact, month, day_of_week, poutcome, y

Table B.1 — Complete descriptive statistics (age, duration, campaign, pdays, previous)

Statistic	age	duration	campaign	pdays	previous
count	41188.00	41188.00	41188.00	41188.00	41188.00
mean	40.02	258.28	2.56	962.47	0.17
std	10.42	259.27	2.77	186.91	0.49
min	17.00	0.00	1.00	0.00	0.00
25%	32.00	102.00	1.00	999.00	0.00
50%	38.00	180.00	2.00	999.00	0.00

75%	47.00	319.00	3.00	999.00	0.00
max	98.00	4918.00	56.00	999.00	7.00

Table B.2 — Complete descriptive statistics (economic variables)

Statistic	emp.var.rate	cons.price.idx	cons.conf.idx	euribor3m	nr.employed
count	41188.00	41188.00	41188.00	41188.00	41188.00
mean	0.08	93.57	-40.50	3.62	5167.03
std	1.57	0.57	4.62	1.73	72.25
min	-3.40	92.20	-50.80	0.63	4963.60
25%	-1.80	93.07	-42.70	1.34	5099.10
50%	1.10	93.74	-41.80	4.85	5191.00
75%	1.40	93.99	-36.40	4.96	5228.10
max	1.40	94.76	-26.90	5.04	5228.10

Table B.5 — Complete categorical frequencies

Job

Category	Count
admin.	10422
blue-collar	9254
technician	6743
services	3969
management	2924
retired	1720
entrepreneur	1456
self-employed	1421
housemaid	1060
unemployed	1014

student	875
unknown	330

Marital

Category	Count
married	24928
single	11568
divorced	4612
unknown	80

Education

Category	Count
university.degree	12168
high.school	9515
basic.9y	6045
professional.course	5243
basic.4y	4176
basic.6y	2292
unknown	1731
illiterate	18

Default

Category	Count
no	32588
unknown	8597
yes	3

Housing

Category	Count

yes	21576
no	18622
unknown	990

Loan

Category	Count
no	33950
yes	6248
unknown	990

Contact

Category	Count
cellular	26144
telephone	15044

Month

Category	Count
may	13769
jul	7174
aug	6178
jun	5318
nov	4101
apr	2632
oct	718
sep	570
mar	546
dec	182

Day of week

Category	Count
thu	8623
mon	8514
wed	8134
tue	8090
fri	7827

Poutcome

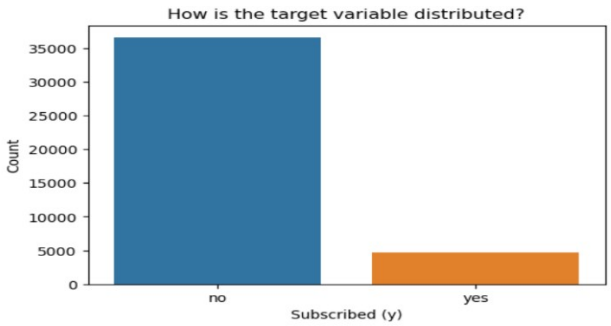
Category	Count
nonexistent	35563
failure	4252
success	1373

y

Category	Count
no	36548
yes	4640

Table B.6 — Target variable distribution

Class	Count	Percentage (%)
no	36548	88.73
yes	4640	11.27



Visual Display of Class imbalance

Table B.7 — Data quality checks

Number of duplicate rows: 12.

Missing values per column: None detected.

Column	Count
age	0
job	0
marital	0
education	0
default	0
housing	0
loan	0
contact	0
month	0
day_of_week	0
duration	0
campaign	0
pdays	0
previous	0
poutcome	0

emp.var.rate	0
cons.price.idx	0
cons.conf.idx	0
euribor3m	0
nr.employed	0
y	0

'Unknown' values per column

Column	Count
job	330
marital	80
education	1731
default	8597
housing	990
loan	990

Figure B.8 — Numerical feature distributions (Group A): age, duration, campaign, pdays, previous, emp.var.rate

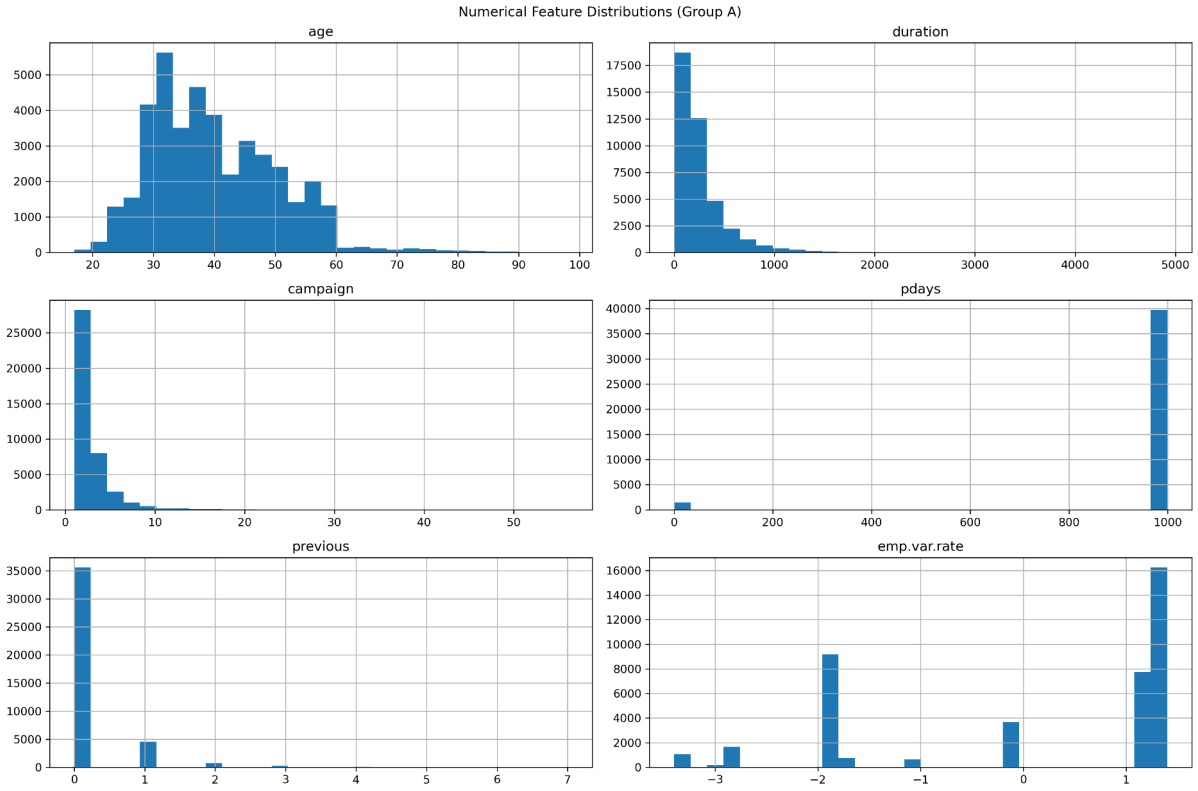


Figure B.8.1 — Numerical feature distributions *(Group B):* cons.price.idx, cons.conf.idx, euribor3m, nr.employed.

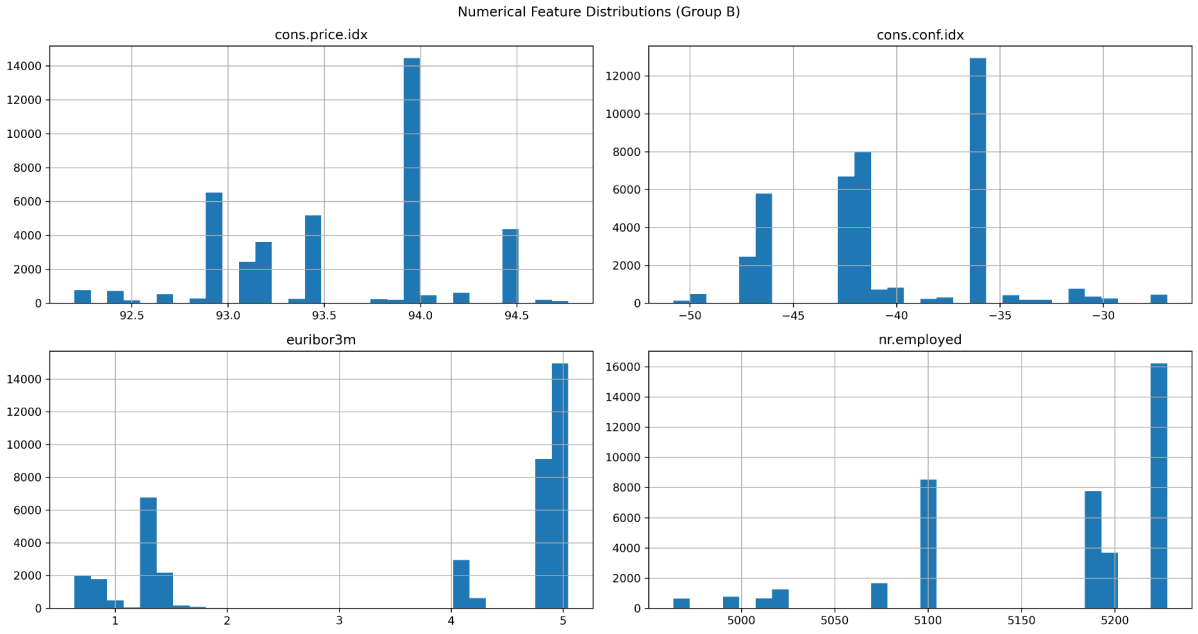


Figure B.8.2 — Boxplots for numerical feature Outlier Detection (: age, duration, campaign, pdays, previous, emp.var.rate.

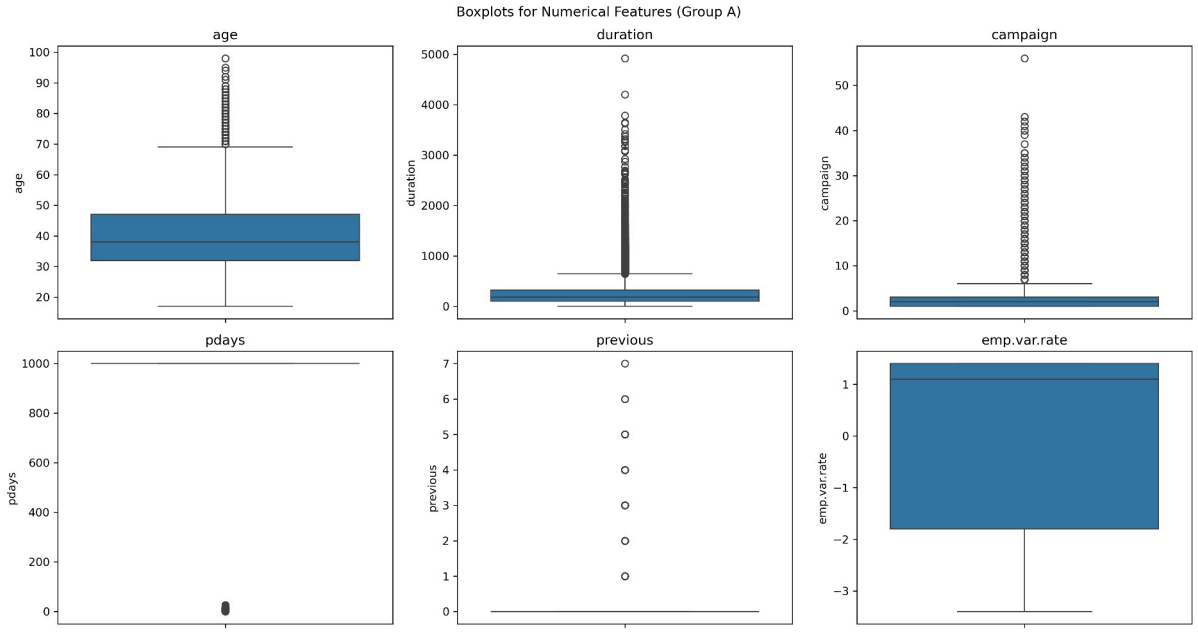


Figure B.8.3 — Boxplots for numerical features (Group B): cons.price.idx, cons.conf.idx, euribor3m, nr.employed

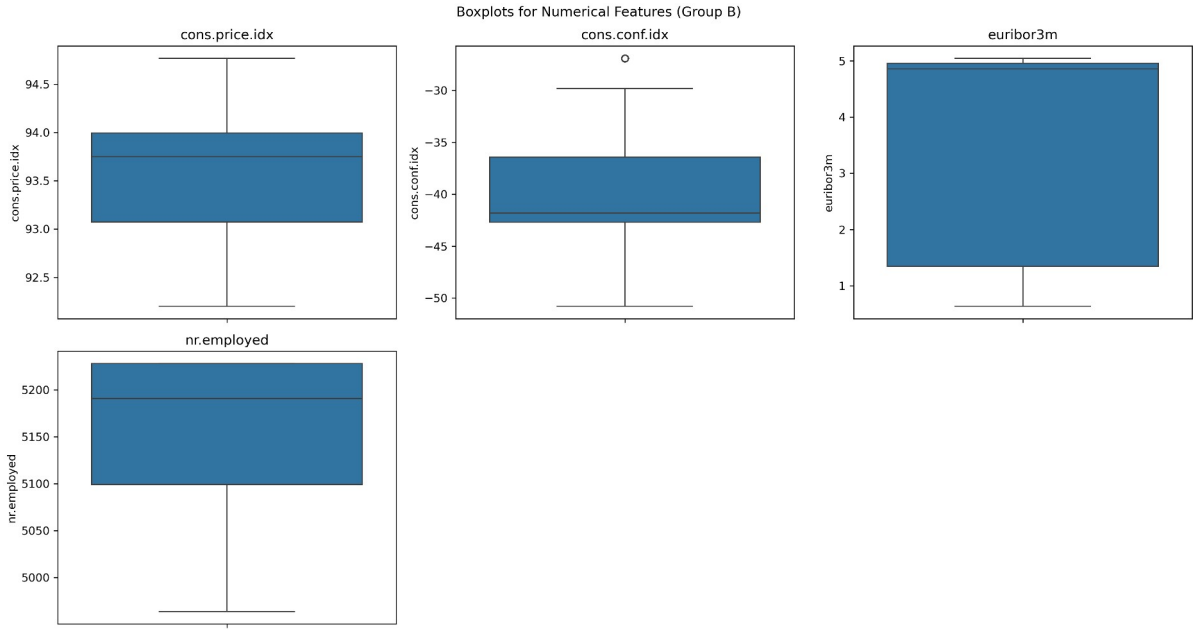


Figure B.9 — Correlation heatmap — numerical features

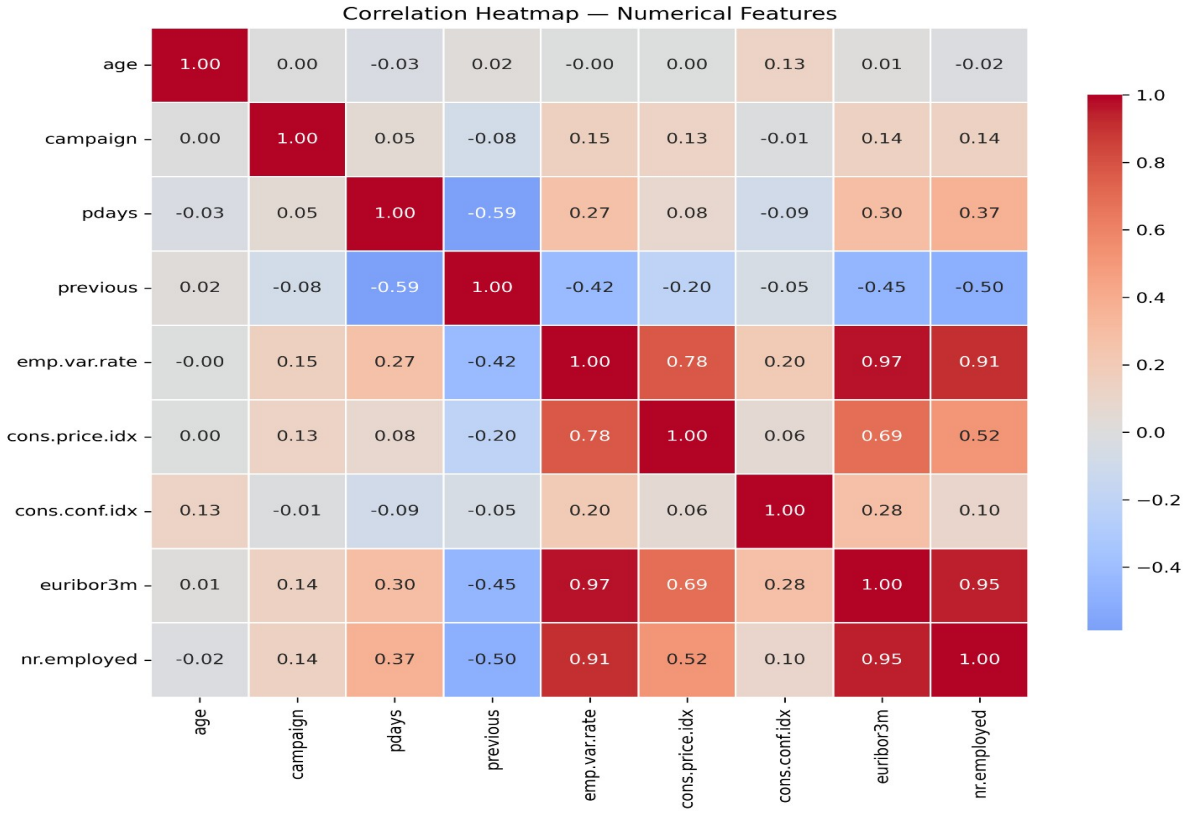


Figure B.10 — Numerical feature differences by subscription outcome

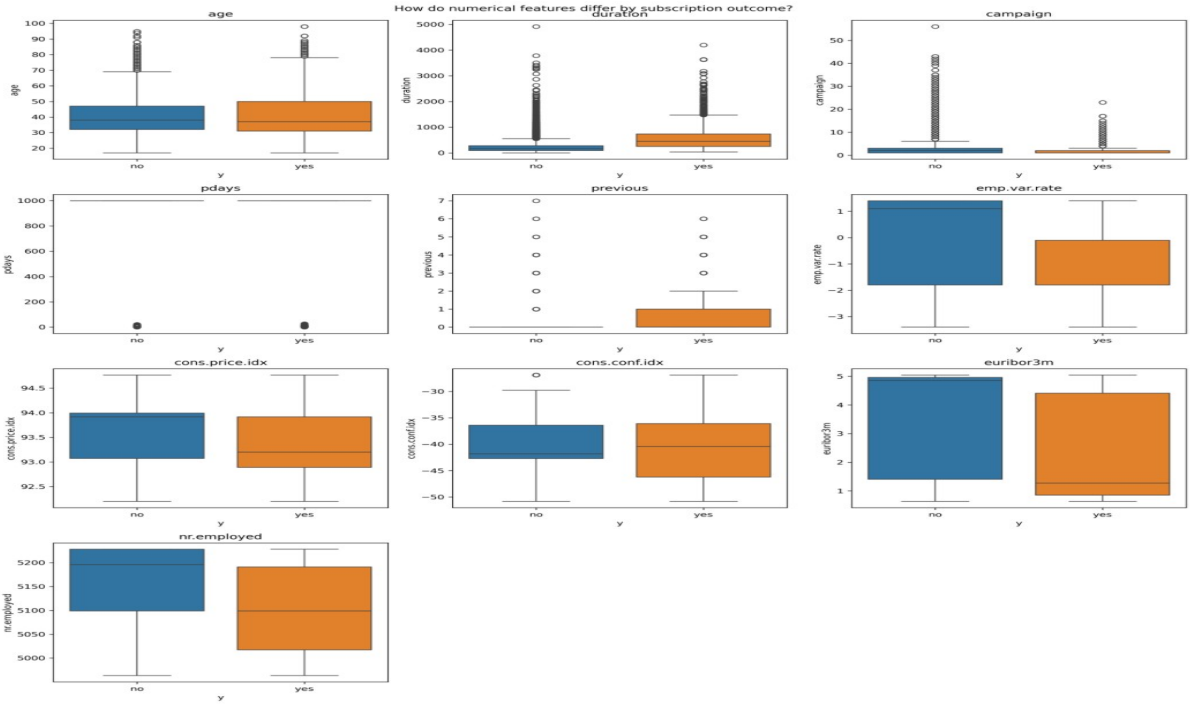


Table B.11 — Subscription rates (%) by categorical variable

Job

Category	Subscription rate (%)
student	31.43
retired	25.23
unemployed	14.20
admin.	12.97
management	11.22
unknown	11.21
technician	10.83
self-employed	10.49
housemaid	10.00
entrepreneur	8.52
services	8.14
blue-collar	6.89

Marital

Category	Subscription rate (%)
unknown	15.00
single	14.00
divorced	10.32
married	10.16

Education

Category	Subscription

	rate (%)
illiterate	22.22
unknown	14.50
university.degree	13.72
professional.course	11.35
high.school	10.84
basic.4y	10.25
basic.6y	8.20
basic.9y	7.82

Default

Category	Subscription rate (%)
no	12.88
unknown	5.15
yes	0.00

Housing

Category	Subscription rate (%)
yes	11.62
no	10.88
unknown	10.81

Loan

Category	Subscription rate (%)
no	11.34
yes	10.93

unknown	10.81

Contact

Category	Subscription rate (%)
cellular	14.74
telephone	5.23

Month

Category	Subscription rate (%)
mar	50.55
dec	48.90
sep	44.91
oct	43.87
apr	20.48
aug	10.60
jun	10.51
nov	10.14
jul	9.05
may	6.43

Day of week

Category	Subscription rate (%)
thu	12.12
tue	11.78
wed	11.67

fri	10.81
mon	9.95

Poutcome

Category	Subscription rate (%)
success	65.11
failure	14.23
nonexistent	8.83

APPENDIX C: COMPLETE PREPROCESSING OUTPUTS

This appendix presents the complete data preprocessing outputs for the Bank Marketing dataset (Moro et al., 2014). Only the interpreted findings arising from this preprocessing are discussed in the main chapters; the full outputs are provided here for verification and reproducibility.

Table C.1 – Duplicate removal summary

Metric	Value
Rows before	41188
Rows after	41176
Duplicates removed	12

Table C.2 – Drop duration (data leakage)

Metric	Value
Shape after removal	(41176, 20)
'duration' in columns	False

Table C.3 – Feature engineering: pdays → previously_contacted

previously_contacted	Count
0	39661
1	1515

'pdays' in columns: False

Table C.4 – Target variable encoding

Before encoding

y	Count
no	36537
yes	4639

After encoding

y	Count
0	36537
1	4639

Class proportions

y	Proportion
0	0.887337
1	0.112663

Missing values in y: 0

Table C.5 – Separate predictors and target

Metric	Value
X shape	(41176, 19)
y shape	(41176,)

Predictor columns:

age, job, marital, education, default, housing, loan, contact, month, day_of_week, campaign, previous, poutcome, emp.var.rate, cons.price.idx, cons.conf.idx, euribor3m, nr.employed, previously_contacted

Possible existing dummy variables: previously_contacted

Table C.6 – Train-validation-test data split shapes (60/20/20, stratified)

Dataset	Shape
X_train	(24705, 19)
X_validation	(8235, 19)
X_test	(8236, 19)
y_train	(24705,)
y_validation	(8235,)
y_test	(8236,)

Table C.7 – Validation of train-test-validation split

Split proportions

Metric	Value
Training proportion	0.6
Validation proportion	0.2
Test proportion	0.2
Total proportion	1.0

Class balance

Split	Class balance
Train	0.11264926128314107
Validation	0.11268973891924712
Test	0.11267605633802817

All checks passed: no overlap, all records accounted for.

Table C.8 – Identify feature types

Numerical features (n = 9)

age, campaign, previous, emp.var.rate, cons.price.idx, cons.conf.idx, euribor3m, nr.employed, previously_contacted

Categorical features (n = 10)

job, marital, education, default, housing, loan, contact, month, day_of_week, poutcome

Table C.9 – Outlier capping (IQR method – fit on train, applied to train and test)

Learned IQR bounds (fit on train)

Feature	Lower bound	Upper bound
age	9.5	69.5
campaign	-2.0	6.0

Summary after clipping (train)

Statistic	age	campaign
count	24705.00	24705.00
mean	39.93	2.27
std	10.16	1.55
min	17.00	1.00
25%	32.00	1.00
50%	38.00	2.00
75%	47.00	3.00
max	69.50	6.00

Table C.10 – One-hot encoding (Branch A – Logistic Regression)

Dataset	Shape
X_train_processed	(24705, 52)
X_validation_processed	(8235, 52)
X_test_processed	(8236, 52)

Example encoded feature names:

['num__age', 'num__campaign', 'num__previous', 'num__emp.var.rate', 'num__cons.price.idx']

Last 5 feature names: ['cat__day_of_week_thu', 'cat__day_of_week_tue', 'cat__day_of_week_wed', 'cat__poutcome_nonexistent', 'cat__poutcome_success']

All checks passed.

Table C.11 – Feature scaling (Branch A – Logistic Regression)

Dataset	Shape
X_train_lr	(24705, 52)
X_validation_lr	(8235, 52)
X_test_lr	(8236, 52)

Table C.12 – Ordinal encoding (Branch B – Random Forest and XGBoost)

Dataset	Shape
X_train_tree	(24705, 19)
X_validation_tree	(8235, 19)
X_test_tree	(8236, 19)

Table C.13 – Final preprocessing validation

Missing values – LR branch

Split	Missing values
Train	0
Validation	0
Test	0

Missing values – Tree branch

Split	Missing values
Train	0
Validation	0
Test	0

Column consistency

Check	Result
LR columns match	True
Tree columns match	True

Row counts match y – LR branch

Split	Match
Train	True
Validation	True
Test	True

Row counts match y – Tree branch

Split	Match
Train	True
Validation	True
Test	True

APPENDIX D: MACHINE-LEARNING OUTPUTS

This appendix presents the complete machine learning model outputs referenced throughout the methodology and results chapters. Only interpreted findings are discussed in the main text; the full metric outputs are provided here for verification and reproducibility.

Logistic Regression

Table D.1: 5-fold cross-validation (mean ± standard deviation)

Metric	Mean	Std. dev.
Accuracy	0.8254	0.0051
Precision	0.3461	0.0102
Recall	0.6180	0.0219
F1	0.4436	0.0120
ROC-AUC	0.7821	0.0069

Table D.2: GridSearchCV — best parameters and best CV score

Best parameters

Parameter	Value
classifier__C	10
classifier__penalty	l2
classifier__solver	liblinear

Best CV F1: 0.44391443794227065

Validation set (chosen model)

Metric	Value
Accuracy	0.82829386763813
Precision	0.35290556900726394
Recall	0.6282327586206896
F1	0.45193798449612405
ROC-AUC	0.800315474533159

Table D.3: Final model — training, validation, and test metrics

Training set (final model)

Metric	Value
Accuracy	0.8261485529245092
Precision	0.3474576271186441
Recall	0.6187567373338124
F1	0.44501873627083605
ROC-AUC	0.7883481426963654

Validation set (final model)

Metric	Value
Accuracy	0.82829386763813
Precision	0.35290556900726394
Recall	0.6282327586206896
F1	0.45193798449612405
ROC-AUC	0.800315474533159

Test set (final model, evaluated once)

Metric	Value
Accuracy	0.8298931520155415
Precision	0.358974358974359
Recall	0.6487068965517241

F1	0.4621880998080614
ROC-AUC	0.7964545378942304

Random Forest

Table D.4: 5-fold cross-validation (mean ± standard deviation)

Metric	Mean	Std. dev.
Accuracy	0.8899	0.0035
Precision	0.5166	0.0226
Recall	0.3450	0.0196
F1	0.4136	0.0205
ROC-AUC	0.7703	0.0183

Table D.5: GridSearchCV — best parameters and best CV score

Best parameters

Parameter	Value
classifier__max_depth	10
classifier__min_samples_leaf	2
classifier__min_samples_split	5
classifier__n_estimators	200

Best CV F1: 0.4779135067277885

Validation set (chosen model)

Metric	Value
Accuracy	0.8769884638737098
Precision	0.4608294930875576
Recall	0.5387931034482759
F1	0.49677098857426727
ROC-AUC	0.7991321500875401

Table D.6: Final model — training, validation, and test metrics

Training set (final model)

Metric	Value
Accuracy	0.8870269176280106
Precision	0.4987413467589679
Recall	0.5695292849443047
F1	0.5317899681261533
ROC-AUC	0.8367543218184172

Validation set (final model)

Metric	Value
Accuracy	0.8769884638737098
Precision	0.4608294930875576
Recall	0.5387931034482759
F1	0.49677098857426727
ROC-AUC	0.7991321500875401

Test set (final model, evaluated once)

Metric	Value
Accuracy	0.8797960174842157
Precision	0.4707547169811321
Recall	0.5377155172413793
F1	0.5020120724346077
ROC-AUC	0.7976511923635883

XGBoost

Table D.7: 5-fold cross-validation (mean ± standard deviation)

Metric	Mean	Std. dev.
Accuracy	0.8954	0.0035
Precision	0.5676	0.0296
Recall	0.3033	0.0185
F1	0.3950	0.0199
ROC-AUC	0.7675	0.0099

Table D.8: GridSearchCV — best parameters and best CV score

Best parameters

Parameter	Value
classifier__learning_rate	0.01
classifier__max_depth	5
classifier__n_estimators	200
classifier__subsample	1.0

Best CV F1: 0.48143846292357334

Validation set (chosen model)

Metric	Value
Accuracy	0.8763812993321191
Precision	0.4585635359116022
Recall	0.5366379310344828
F1	0.4945382323733863
ROC-AUC	0.7948993761296441

Table D.9: Final model — training, validation, and test metrics

Training set (final model)

Metric	Value
Accuracy	0.8754503137016798
Precision	0.45493562231759654
Recall	0.5332375134746676
F1	0.4909842845326716
ROC-AUC	0.7883556661856332

Validation set (final model)

Metric	Value
Accuracy	0.8763812993321191
Precision	0.4585635359116022
Recall	0.5366379310344828
F1	0.4945382323733863
ROC-AUC	0.7948993761296441

Test set (final model, evaluated once)

Metric	Value
Accuracy	0.8817387081107334
Precision	0.47821969696969696
Recall	0.5441810344827587
F1	0.5090725806451613
ROC-AUC	0.7966666489723119

Model Comparison

Table D.10: Model comparison across Logistic Regression, Random Forest, and XGBoost

Validation performance

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.8283	0.3529	0.6282	0.4519	0.8003
Random Forest	0.8770	0.4608	0.5388	0.4968	0.7991
XGBoost	0.8764	0.4586	0.5366	0.4945	0.7949

Test performance (final)

Model	Accuracy	Precision	Recall	F1	ROC-AUC
XGBoost	0.8817	0.4782	0.5442	0.5091	0.7967
Random Forest	0.8798	0.4708	0.5377	0.5020	0.7977
Logistic Regression	0.8299	0.3590	0.6487	0.4622	0.7965

Final ranking (F1 + ROC-AUC average)

Model	F1	ROC-AUC	F1 + ROC-AUC (avg)
XGBoost	0.5091	0.7967	0.6529
Random Forest	0.5020	0.7977	0.6498
Logistic Regression	0.4622	0.7965	0.6294

Validation–test gap

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	-0.0016	-0.0061	-0.0205	-0.0103	0.0038
Random Forest	-0.0028	-0.0100	0.0011	-0.0052	0.0014
XGBoost	-0.0053	-0.0196	-0.0076	-0.0146	-0.0018

Final selected model: XGBoost
SHAP Analysis

Figure D.11 — SHAP feature importance (mean |SHAP value|) — XGBoost

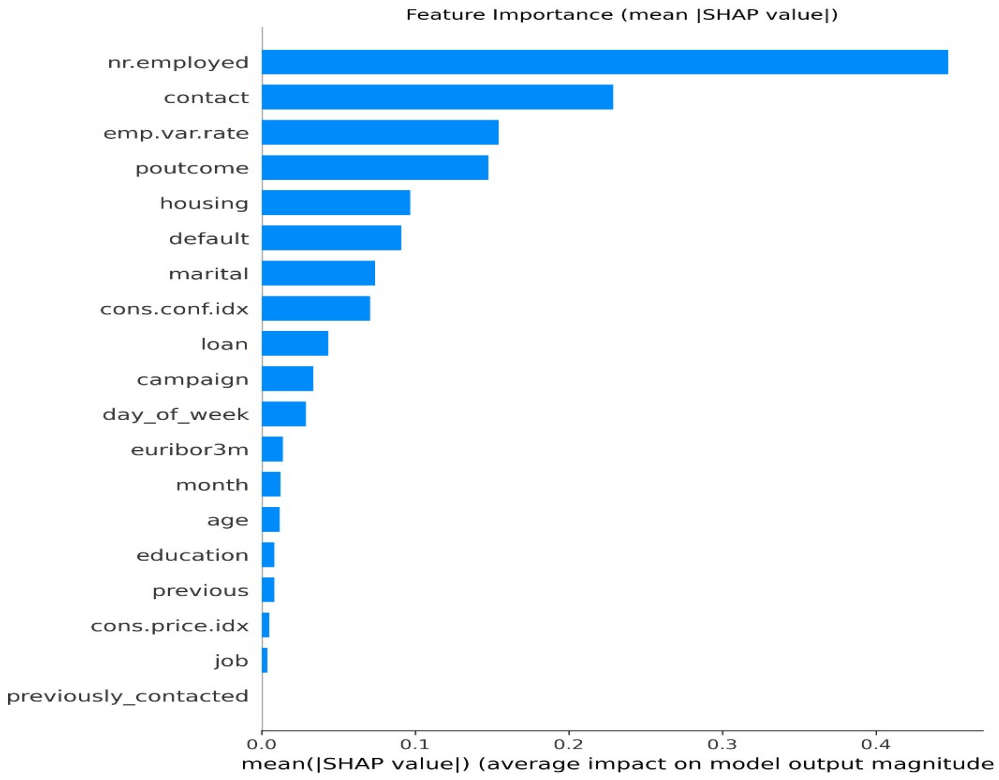


Table D.12 — SHAP feature importance ranking

Feature	Mean |SHAP|	Rank
nr.employed	0.447066	1
contact	0.228964	2
emp.var.rate	0.154315	3
poutcome	0.147819	4
housing	0.096710	5
default	0.090980	6
marital	0.073959	7

cons.conf.idx	0.070681	8
loan	0.043288	9
campaign	0.033532	10
day_of_week	0.028919	11
euribor3m	0.013738	12
month	0.012434	13
age	0.011914	14
education	0.008215	15
previous	0.008199	16
cons.price.idx	0.005150	17
job	0.003601	18
previously_contacted	0.000140	19

Figure D.2 — SHAP summary plot — feature impact on subscription

APPENDIX E: TESTING AND VERIFICATION EVIDENCE

Table G.1 presents the testing and verification activities conducted throughout the development of the machine learning artefact. The objectives of these checks were to test, validate, and document the implementation sequence, beginning from data acquisition, exploratory data analysis, preprocessing, model development, optimisation, evaluation and explainability stages function as intended.

Table 1 G.1: Detailed outputs are cross-referenced to the relevant appendix tables and figures.

Test ID	Testing / Verification Step	What Was Verified	Expected Outcome	Evidence / Cross-Reference	Result
T1	**Dataset Loading**	The Portuguese Bank Marketing dataset was successfully imported into Jupyter Notebook.	Dataset consist of 41,188 observations and 21 variables.	Appendix B, Table B.1: Appendix A, Listings A1.1, A1.2, A1.3.	**Pass**
T2	**Data Quality Assessment**	Duplicate records, missing values and 'unknown' categorical values were identified.	Data-quality issues are successfully detected before preprocessing.	Appendix B, Table B.7: Appendix A, Listing A2.4.	**Pass**
T3	**Cleaning, Feature Engineering & Target Encoding**	The removal of duplicates and duration feature. Transforming "pdays" to previously_contacted, and encoding of the target variable y from yes/no to 1/0 were completed succesfully.	12 duplicates and duration removed; previously_contacted transformed; target contains only binary values without changing the original class distribution.	Appendix C, Tables C.1–C.5: Appendix A, Listings A3.2–A3.6.	**Pass**
T4	**Stratified Dataset Partitioning**	Dataset was partitioned using the 60/20/20 data split.	All observations are accounted for with approximately similar subscriber proportion in each train, validate and test subset.	Appendix C, Tables C.6–C.7: Appendix A, Listings A3.7–A3.8.	**Pass**
T5	**Leakage / Dataset Isolation**	Training, validation and test subsets remained separate during model development.	No overlap between data splits; preprocessing parameters learned from training data only; test set isolated from model development.	Appendix C, Table C.7: Appendix A, Listing A3.8; Chapter 5 methodology.	**Pass**
T6	**Outlier Treatment**	IQR thresholds were learned from training data and consistently applied to validation/test dataset throughout iterations.	Outliers were capped without removing customer observations or using validation/test data to determine the thresholds.	Appendix C, Table C.9: Appendix A, Listing A3.10.	**Pass**
T7	**Logistic Regression Preprocessing**	One-Hot Encoding and StandardScaler was successfully applied on Logistic Regression model.	Categorical features were encoded. Logistic required numerical variables standardised.	Appendix C, Tables C.10–C.11: Appendix A, Listings A3.11–A3.12.	**Pass**
T8	**Random Forest and XGBoost Preprocessing**	Ordinal Encoding was successfully applied on Random Forest and XGBoost while numerical features remained unscaled.	Training, validation and test subsets remain consistent with 19-predictor.	Appendix C, Table C.12: Appendix A, Listing A3.13.	**Pass**
T9	**Preprocessing Validation**	Final numerical and categorical features, duplicates, missing values, row counts and target variable were checked post-transformation.	9 numerical and 10 categorical predictors identified. no unexpected duplicates or missing values. Consistent feature structure. predictor correctly aligned.	Appendix C, Tables C.8, C.13: Appendix A, Listings A3.9, A3.14.	**Pass**
T10	**SMOTE Pipeline**	SMOTE was integrated within the model-development pipeline rather than before dataset partitioning.	Synthetic oversampling restricted to training folds and not applied to validation/test data.	Appendix A, Listings A4.1, A4.2.1, A4.3.1.	**Pass**
T11	**Five-Fold Stratified Cross-Validation**	All three algorithms, Logistic Regression, Random Forest and XGBoost was successfully implemented within the common stratified CV	Five stratified folds successfully evaluated each model.	Appendix D, Tables D.1, D.4, D.7: Appendix A, Listings A4.1, A4.2.1, A4.3.1.	**Pass**

Test ID	Testing / Verification Step	What Was Verified	Expected Outcome	Evidence / Cross-Reference	Result
		framework.			
T12	**GridSearchCV Optimisation**	Hyperparameter optimisation was successfully implemented for each classifier.	Optimal parameter combinations obtained using training data and five-fold stratified cross-validation.	Appendix D, Tables D.2, D.5, D.8: Appendix A, Listings A4.2, A4.2.2, A4.4.2.	**Pass**
T13	**Validation & Model Comparison**	Optimised models produced predictions on the validation set and was compared using consistent data boundaries and evaluation metrics.	Accuracy, Precision, Recall, F1-score and ROC-AUC were successfully measured for all three models, providing consistent model comparison.	Appendix D, Table D.10 (validation performance); Appendix A, Listing A5.1.	**Pass**
T14	**Independent Test Evaluation**	Final models produced predictions on the isolated test set.	Final performance metrics was successfully measured using unseen test observations.	Appendix D, Tables D.3, D.6, D.9, D.10 (test performance); Appendix A, Listings A4.4, A4.2.4, A4.4.4.	**Pass**
T15	**SHAP EAI**	SHAP analysis successfully generated global feature-importance values for the selected algorithm.	Valid SHAP values and a ranked global feature-importance output was generated for interpretation in the Discussion chapter.	Appendix D, Figure D.13, Table D.12; Appendix A, Listing A6.1.	**Pass**
T16	**Reproducibility**	Random states, required library versions, pipeline sequence and notebook implementation was checked.	The interactive experimental workflow contains the documented technical environment, dataset information, fixed random states and source code required to support reproducible production.	Appendix A, Listing A1.1 (environment/library imports); fixed random_state=42 throughout Appendix A (e.g. Listings A3.7, A4.1–A4.4.4).	**Pass**

APPENDIX F: RISK REGISTER

Table F.1 Presents the risks identified during the dissertation project, their category, likelihood, impact and resulting risk score, the week each risk was identified, the mitigation applied, and current status. Likelihood and impact are rated as Low = 1, Medium = 2 and High = 3. The overall risk score is calculated as Likelihood × Impact, producing a score between 1 and 9. Status reflects the project's position at the time of writing.

ID	Risk	Category	Likelihood	Impact	Score	Identified	Mitigation	Status
R1	Class imbalance may influence models to favour non-subscribers and inflate Accuracy.	Data / Modelling	High	High	**9**	Wk 2	SMOTE was applied and restricted to training folds, while F1-score and ROC-AUC were prioritised for model comparison.	Mitigated
R2	Data leakage across training, validation and test splits could generate unrealistic model performance.	Data	Medium	High	**6**	Wk 3	Validation and test data remained isolated, and preprocessing parameters were learned from training data only.	Mitigated
R3	Models may overfit the training data and generalise poorly.	Modelling	Medium	High	**6**	Wk 5	Used five-fold stratified cross-validation, GridSearchCV and independent test evaluation.	Mitigated
R4	duration is only available after the call and could influence unrealistic pre-contact predictions.	Data	High	High	**9**	Wk 3	Removed duration prior to model development.	Resolved
R5	Results may become difficult to reproduce.	Technical / Process	Medium	Medium	**4**	Wk 1	Implemented fixed random states, consistent pipelines, required library version, relative file paths and documented jupyter notebook code.	Mitigated
R6	Complex ensemble models may be difficult to interpret for a general marketing audience.	Modelling / Business	High	Medium	**6**	Wk 6	Applied SHAP to the selected model to identify and rank influential features associated with the dataset.	Mitigated
R7	Exceeding the dissertation word count could require heavy editing.	Project Management / Academic	Medium	High	**9**	Wk 12	Continuously monitored word counts, moved detailed technical outputs to appendices, and prioritised content directly linked to the research aims and objectives.	Ongoing
R8	Remaining project time and could prevent sufficient evaluation of an additional Decision Tree model alongside LR, RF and XGBoost.	Project Management	High	High	**9**	Wk 8	Reviewed the remaining schedule timeline and removed Decision Tree from the final model comparison, only prioritising LR, RF, XGBoost, cross-model evaluation, SHAP and dissertation writing.	Resolved
R9	An unexpected disruption resulted in missed supervisor contact between 10–31 July and reduced project output, creating a risk of falling behind planned schedule.	Project Management	High	High	**9**	Wk 2	Supervisor contact was resumed, additional development time was allocated, and delayed EDA, preprocessing and modelling activities were recovered without changing the final submission deadline.	Resolved
R10	Insufficient relevant literature review could weaken the justification of methodological decisions and comparison of findings with previous research.	Academic / Literature	Medium	High	**6**	Wk 6	Following supervisor meeting feedback, a focused literature review identified comparable bank-marketing and machine-learning studies that were cross-referenced during the project construction and dissertation writing.	Mitigated

APPENDIX G: PROJECT MILESTONE TABLE

Table G.1 Illustrates the dissertation's project milestones across the 12-week timeline, distinguishing completed, ongoing and planned activities, and notes significant events affecting the schedule. Decision Tree modelling was removed from scope at Week 8 due to time constraints (Risk R8). A period of missed supervisor contact and schedule disruption (Risk R9), together with an extension of the literature review following supervisor feedback on model evaluation (Risk R10), are noted against the milestones they affected.

Milestone	Target Week	Actual Week	Status	Notes
Project Definition / Business Problem Identification	Wk 1	Wk 1	**Completed**	
Initial Literature Review	Wk 1–2	Wk 1–2	**Completed**	Ongoing cross-referencing continued through Wk 8 (see R10).
Requirements Analysis	Wk 1	Wk 1	**Completed**	
System / Experimental Design	Wk 2	Wk 2	**Completed**	Included confirmation of the 60/20/20 train/validation/test split.
Data Acquisition & Cleaning	Wk 1	Wk 1	**Completed**	
Exploratory Data Analysis	Wk 2	Wk 2	**Completed**	Overlapped with the start of the schedule disruption (R9).
Data Preprocessing Pipeline	Wk 3–4	Wk 3–4	**Completed**	Delayed by schedule disruption (R9); recovered via additional hours.
Logistic Regression: CV → GridSearch	Wk 5	Wk 5	**Completed**	Part of the post-disruption recovery period (R9).
Random Forest: CV → GridSearch	Wk 6	Wk 6	**Completed**	
XGBoost: CV → GridSearch	Wk 7	Wk 7	**Completed**	
Extended Literature Review: Evaluation Metrics	Wk 8	Wk 8	**Completed**	Added following supervisor feedback (R10); Decision Tree removed from scope this week (R8).
Cross-Model Comparison (F1-score & ROC-AUC)	Wk 9	Wk 9	**Completed**	
SHAP Analysis (winning model)	Wk 10	Wk 10	**Completed**	
CRISP-DM Narrative & Write-up	Wk 11–12	Wk 11–12	**Completed**	
Review, Proofread & Formatting	Wk 13	Wk 9–(ongoing)	**Ongoing**	
Final Submission	Wk 14	—	**Milestone**	